

Learning Difficulty-Aware Length Control for Efficient Hybrid Reasoning Models

Jaejun Shim¹
junshim@skku.edu

HyunJin Kim¹
khyunjin1993@skku.edu

Young Jin Kim² *
youki@microsoft.com

JinYeong Bak¹ *
jy.bak@skku.edu

¹Sungkyunkwan University ²Microsoft

Abstract

Large Reasoning Models (LRMs) achieve strong performance on complex tasks but exhibit systematic inefficiency: they often *overthink* easy problems and *underthink* hard ones. Existing approaches based on uniform length penalties or rigid routing incur an *efficiency tax*, trading reduced computation on easy instances for accuracy loss on hard instances. We formulate efficient reasoning as an instance-adaptive computation allocation problem and propose **When2Think**, a post-training framework for hybrid reasoning that dynamically allocates computation based on problem difficulty. Our method introduces **Instance-level Difficulty-Aware Control (IDAC)**, a reward-shaping mechanism that leverages pre-computed reference statistics (accuracy and token usage) to regulate reasoning depth. Combined with verifier-based rewards and batch-wise standardized advantages, IDAC enables stable critic-free optimization without learned reward models or online reference-model queries. *When2Think* encourages direct answering on easy instances while preserving extended reasoning on hard instances, thereby learning when to use SYSTEM 1 (NOTHINK) versus SYSTEM 2 (THINK). Experiments on mathematical benchmarks demonstrate improved accuracy–efficiency trade-offs: on AIME24, Pass@3 increases by 10.0% while token usage is reduced by 27.9% relative to the base model, and on AIME25, *When2Think* achieves 40.0% Pass@3, outperforming compression and routing-only baselines.

1 Introduction

The emergence of *large language models (LLMs)* [1–5] has transformed *artificial intelligence (AI)*, enabling fluent and coherent text generation across a wide range of tasks. However, many tasks of interest require not only fluent text generation but also reliable multi-step reasoning [6, 7]. A prominent direction is to internalize structured reasoning capabilities through *supervised fine-tuning (SFT)* or *reinforcement fine-tuning (RFT)* [8–10], producing **large reasoning models (LRMs)** that generate coherent reasoning trajectories and achieve strong performance on challenging benchmarks [11–13].

Despite these advances, LRMs suffer from *reasoning inefficiency* [14]. They often **overthink** simple tasks by generating redundant reasoning tokens without accuracy gains [15, 16], yet **underthink** complex tasks by terminating reasoning prematurely or producing fragmented chains [17]. Static token-reduction strategies can further introduce an **efficiency tax**: suppressing overthinking on easy instances systematically harms performance on harder ones (Table 1).

*Corresponding author

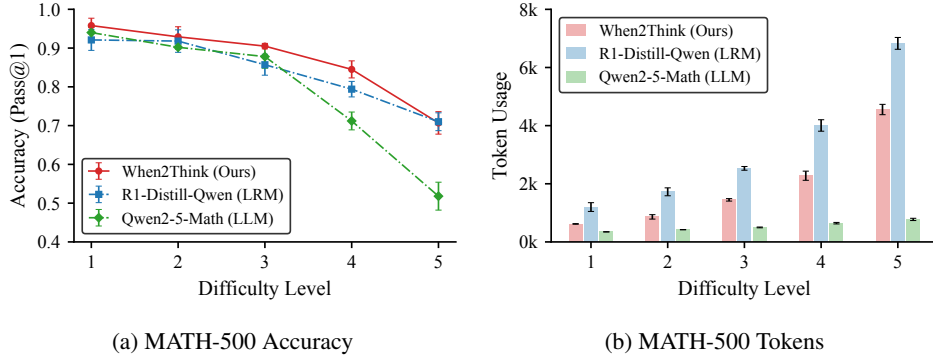


Figure 1: **Performance and efficiency on the MATH-500 benchmark** stratified by problem difficulty (Levels 1–5). We compare the *When2Think* (red), LRM baseline (blue) and LLM baseline (green). **Lines:** Mean Pass@1 accuracy $\pm$ standard deviation. **Bars:** Mean tokens per instance $\pm$ standard deviation. Statistics computed over five independent runs. *When2Think* mitigates the efficiency tax by selectively allocating computation across easy and hard instances.

We adopt *dual-process theory* [18, 19] as a conceptual lens for this mismatch. SYSTEM 1 denotes fast and intuitive responses, whereas SYSTEM 2 denotes slower and more deliberate reasoning [13]. Easy instances are often solvable with SYSTEM 1-style direct answering, while hard instances benefit from SYSTEM 2-style deliberation [20]. As shown in Figure 1, LLMs are efficient on easy instances, whereas LRMs allocate substantially more tokens to improve performance on harder ones.

Motivated by these observations, we propose a post-training framework for *hybrid reasoning*, combining the efficiency of SYSTEM 1-style direct answering with the deliberative power of SYSTEM 2-style reasoning in a single model. We make the following contributions:

- **Unified analysis of reasoning inefficiency.** We analyze the *efficiency tax* (§ 6.1), showing that token reduction reduces overthinking on easy instances but induces underthinking on hard ones.
- **Reasoning as instance-adaptive control.** We recast efficient reasoning as an instance-level adaptive control problem (§ 6.2), shifting the focus from token reduction to difficulty-aware allocation.
- **Adaptive hybrid reasoning via *When2Think*.** We introduce *When2Think* (§ 4), a post-training method that uses Instance-level Difficulty-Aware Control (IDAC)-based reward shaping to control both whether to think and how much to think within a unified hybrid reasoning policy.

Figure 1 previews the behavior learned by *When2Think*, which achieves LLM-level efficiency on easy instances while preserving LRM-level accuracy on harder ones. *When2Think* shows the effectiveness of adaptive reasoning depth on the most challenging AIME benchmarks.

2 Related Work

We survey prior work on reducing reasoning cost in LRMs, focusing on post-training methods for efficient multi-step reasoning. Following recent surveys [14, 16], these approaches fall into two categories: (1) *reasoning compression*, which shortens intermediate reasoning traces, and (2) *hybrid reasoning*, which dynamically balances direct answers and explicit deliberation.

Reasoning Compression. Reasoning compression aims to shorten reasoning trajectories while maintaining correctness. Data-level approaches, such as chunk-level distillation (*Skip-Thinking* [21]) and validity-aware selection (*LC-R1* [22]), remove redundant steps from generated chains. Reinforcement-learning-based techniques introduce efficiency incentives via hard token budgets (*ThinkPrune* [23]) or reward-based length penalties (*LASER* [24], *DLER* [25]). These approaches effectively reduce overthinking on easy instances, but their static or globally tuned constraints can lead to underthinking on hard instances.

Hybrid Reasoning. Hybrid reasoning methods reduce computation by selectively applying reasoning. Reinforcement-learning approaches such as ARM [26] and LHRM [27] switch between short and long reasoning trajectories, while other methods (AdaptThink [28], ThinkLess [29]) skip reasoning on easy instances using learned controllers. Although these methods reduce overthinking or enable hybrid reasoning, they lack fine-grained, instance-level control. To address this limitation, we introduce *When2Think*, which unifies efficient SYSTEM 1 and deliberative SYSTEM 2 reasoning.

3 Preliminaries

Post-training is one effective mechanism for encouraging SYSTEM 2-style reasoning in LLMs by modifying model parameters, leading to models commonly referred to as *Large Reasoning Models (LRMs)* or *Reasoning Language Models (RLMs)* [30–34]. We focus on *reinforcement learning (RL)* approaches within the broader paradigm of *learning to reason* [8, 10, 35], which directly optimize reasoning policies to enhance both accuracy and efficiency [11, 12, 36–38]. Additional reasoning paradigms for LLMs are discussed in § G.

Verifiable Rewards. Lambert et al. [39] introduce *Reinforcement Learning with Verifiable Rewards (RLVR)*, which replaces learned reward models with deterministic verification, directly aligning the RL objective with task correctness and reducing reward-model bias. Prior work [40, 41] uses verifiers either as *Outcome-supervised Reward Models (ORMs)* for final answers or *Process-supervised Reward Models (PRMs)* for step-level supervision, trading simplicity for annotation cost. RLVR leverages outcome-based verification to provide reliable rewards for multi-step reasoning without requiring step-level supervision.

Importance Sampling. To mitigate cold-start bias and prevent premature collapse into the THINK mode, we adopt the importance sampling (IS) strategy of AdaptThink [28]. This IS scheme provides the hybrid reasoning setup used throughout training, while our contribution focuses on instance-adaptive control of reasoning depth. Hybrid reasoning is controlled by an explicit mode token (THINK or NOTHINK) prepended to each trajectory. When THINK is selected, explicit reasoning proceeds until a designated end-of-thinking token (EOT); when NOTHINK is selected, the model skips deliberation and directly generates the final answer, similar to the *NoThinking* mechanism of Ma et al. [42].

During data collection, trajectories are sampled from an auxiliary exploration policy π_{IS} , which selects the initial mode token uniformly to enforce balanced exploration, while all subsequent tokens are generated by the current policy π_{θ} . During optimization, collected trajectories are reweighted using importance ratios, yielding unbiased gradient estimates with respect to the target policy despite the modified exploration distribution.

Clipped Surrogate Objective. We optimize the target policy π_{θ} using a PPO-style clipped policy gradient objective following AdaptThink [28], which stabilizes learning of when to invoke explicit reasoning. Although the objective is defined with respect to trajectories generated by π_{θ} , in practice we optimize using samples collected from an auxiliary policy π_{IS} , with importance weighting to ensure consistency with the target policy.

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ denote a dataset of N instances. During training, we sample a mini-batch $\mathcal{B} \subset \mathcal{D}$ of size M and reindex its elements as $\{(x_i, y_i)\}_{i=1}^M$. For each $x_i \in \mathcal{B}$, we sample K reasoning trajectories, denoted by $\{o_{i,k}\}_{k=1}^K$.

The trajectory-level importance ratio:

$$\rho_{i,k} = \frac{\pi_{\theta}(o_{i,k} \mid x_i)}{\pi_{\text{IS}}(o_{i,k} \mid x_i)}, \quad (1)$$

and the clipped surrogate loss:

$$\mathcal{L}(\theta) = -\mathbb{E}_{x_i \sim \mathcal{B}, o_{i,k} \sim \pi_{\text{IS}}(\cdot \mid x_i)} \left[\min(\rho_{i,k} A_{i,k}, \min(\max(\rho_{i,k}, 1 - \epsilon), 1 + \epsilon) A_{i,k}) \right], \quad (2)$$

where $A_{i,k}$ denotes a trajectory-level advantage estimate and ϵ is the PPO clipping parameter.

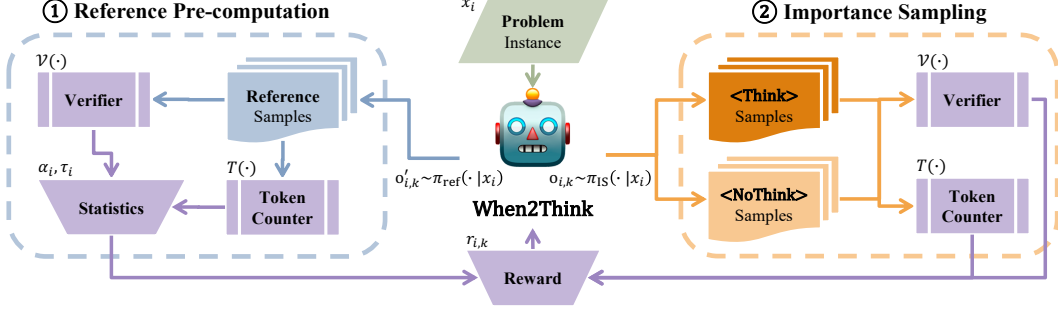


Figure 2: **Overview of the When2Think RLVR framework** for adaptive hybrid reasoning. The framework comprises two stages. **(Left) Reference pre-computation:** A reference policy π_{ref} generates trajectories for each input x_i to estimate instance-level difficulty α_i and expected reasoning cost τ_i . **(Right) Training with importance sampling:** An exploration distribution π_{IS} enforces balanced exploration between THINK (explicit reasoning) and NOTHINK (direct answering) by sampling the initial mode token uniformly, while subsequent tokens are generated by the policy π_{θ} .

4 When2Think

We propose *When2Think*, a post-training reward-shaping framework for adaptive computation allocation in hybrid reasoning. For each input, *When2Think* learns both whether to invoke explicit reasoning (THINK vs. NOTHINK) and how deeply to deliberate, dynamically balancing SYSTEM 1 direct answering and SYSTEM 2 multi-step reasoning. These behaviors are guided by offline reference statistics that estimate instance difficulty and expected reasoning cost (Figure 2).

Rewards jointly favor correctness and efficiency. *Instance-level Difficulty-Aware Control (IDAC)* modulates a correctness-gated efficiency bonus using trajectory length and pre-computed reference statistics: α_i captures instance difficulty through reference accuracy, while τ_i estimates expected reasoning cost. The reward is normalized by the instance-specific baseline α_i and standardized by BWS, enabling stable critic-free PPO-style optimization. Since correctness comes from verifiable rewards and reference statistics are pre-computed offline before each training epoch, training requires no learned reward model, learned critic, or online reference-model queries during policy updates.

4.1 Reference Statistics

To estimate instance difficulty and expected reasoning cost without querying the reference model during inner-loop optimization, we pre-compute epoch-wise reference statistics using a reference policy π_{ref} over the dataset $\mathcal{D}$ and cache them for subsequent policy update steps. For each input $x_i \in \mathcal{D}$, we sample K independent trajectories $o'_{i,k} \sim \pi_{\text{ref}}(\cdot | x_i)$. The final answer is extracted from each trajectory using $g(\cdot)$.

We then define the answer verification function:

$$\mathcal{V}(x_i, o'_{i,k}) = \mathbb{I}[g(o'_{i,k}) = y_i], \quad (3)$$

where $\mathbb{I}[\cdot]$ equals 1 if the extracted answer matches the ground-truth y_i , and 0 otherwise.

Instance Difficulty and Budget. To estimate the difficulty and expected reasoning cost of each instance, we compute the reference accuracy and reference length:

$$\alpha_i = \gamma \left(\frac{1}{K} \sum_{k=1}^K \mathcal{V}(x_i, o'_{i,k}) \right) + \varepsilon, \quad \tau_i = \frac{1}{K} \sum_{k=1}^K \mathcal{T}(o'_{i,k}) + \varepsilon. \quad (4)$$

Here, α_i is the scaled empirical accuracy of the reference policy on instance x_i , serving as a proxy for instance difficulty, while τ_i denotes the average reference trajectory length and serves as an instance-specific token budget. The $\mathcal{T}(\cdot)$ returns the token count of a trajectory and constants $\gamma \in (0, 1)$ and ε control reward scaling and numerical stability, respectively; we use $\gamma = 0.9$ in our experiments.

4.2 Reward Formulation

Our online reward provides fine-grained control over both *whether* to invoke explicit reasoning and *how much* computation to allocate. It comprises three components: (1) Instance-level Difficulty-Aware Control (IDAC) that modulates reasoning cost relative to reference statistics, (2) a hybrid bonus linking SYSTEM 1/2 behavior with length control, and (3) a correctness reward normalized by reference performance. Together, these components enable adaptive instance-level control of reasoning depth. During training, trajectories $o_{i,k}$ are sampled via importance sampling and evaluated using the precomputed reference statistics.

Instance-level Difficulty-Aware Control. We introduce a *Instance-level Difficulty-Aware Control (IDAC)* term to adapt reasoning depth according to instance difficulty.

Formally, it is defined:

$$\tilde{\lambda}_{i,k} = \exp\left(-\frac{\mathcal{T}(o_{i,k})\alpha_i}{\tau_i}\right), \quad \lambda_{i,k} = \begin{cases} \tilde{\lambda}_{i,k} & \text{if Think,} \\ 1 & \text{if NoThink.} \end{cases} \quad (5)$$

Intuitively, this deterministic, trajectory-level factor imposes stronger decay for trajectories that exceed the reference length, discouraging overthinking on easy instances while allowing longer deliberation for harder ones. By operating at the trajectory level rather than token level, IDAC provides stable and interpretable control over reasoning depth. Figure 3 illustrates this behavior.

Instance-Adaptive Reward. The adaptive final reward combines the answer verification function with an instance-specific reference baseline and a correctness-gated efficiency bonus:

$$r_{i,k} = \mathcal{V}(x_i, o_{i,k})(1 + \lambda_{i,k}\delta) - \alpha_i. \quad (6)$$

Here, $\mathcal{V}(x_i, o_{i,k}) \in \{0, 1\}$ is the binary correctness signal provided by the verifier, $\lambda_{i,k}$ is the IDAC scaling factor, and $\delta > 0$ controls the magnitude of the efficiency bonus. The term α_i serves as an instance-specific reference baseline, normalizing rewards relative to the reference policy’s empirical performance on instance x_i .

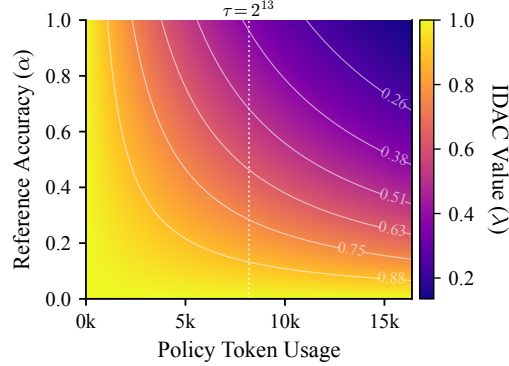


Figure 3: **Visualization of the Instance-level Difficulty-Aware Control (IDAC) mechanism.** The heatmap depicts the scaling factor λ as a function of the trajectory length and the instance-conditioned reference length ($\tau = 2^{13}$, half of the maximum response length 2^{14}).

4.3 Batch-Wise Standardized Advantage

To enable stable trajectory-level credit assignment without a learned critic, we adopt a *batch-wise standardized advantage (BWS)*, drawing upon the stabilization insights of *REINFORCE++* [43].

Given a mini-batch of M instances, we sample K trajectories for each instance i , $\{o_{i,k}\}_{k=1}^K$, and compute a scalar adaptive reward $r_{i,k}$ for each trajectory. For each trajectory index k , we compute the batch-wise mean and standard deviation of rewards across the M instances:

$$\mu_k = \frac{1}{M} \sum_{i=1}^M r_{i,k}, \quad \sigma_k = \sqrt{\frac{1}{M} \sum_{i=1}^M (r_{i,k} - \mu_k)^2}. \quad (7)$$

The trajectory-level advantage is defined:

$$A_{i,k} = \frac{r_{i,k} - \mu_k}{\sigma_k + \varepsilon}, \quad (8)$$

where $\varepsilon > 0$ is a small constant for numerical stability. All tokens in a trajectory share the same standardized advantage, yielding a low-variance estimator that naturally supports trajectory-level credit assignment in the PPO-style clipped surrogate optimization described in § 3.

5 Experimental Setup

We evaluate whether this adaptive control maintains accuracy on difficult instances while reducing unnecessary computation on easier ones. Implementation details are provided in § A.

5.1 Training

We train *When2Think* on a reasoning-capable base model, *R1-distill-1.5B*, a compact language model distilled from *DeepSeek-R1* [32] and built on *Qwen2.5-Math-1.5B* [44]. The model was distilled using approximately 800k curated samples via SFT. Training is conducted on the *DeepScaleR* dataset [45], which contains approximately 40k competition-level mathematics problems with verified solutions. The dataset aggregates publicly available sources, including AIME (1984–2023), AMC (pre-2023), Omni-MATH [46], and Still [47], covering algebra, geometry, number theory, and combinatorics.

5.2 Evaluation

We evaluate models on a diverse suite of mathematical reasoning benchmarks spanning grade-school arithmetic, olympiad challenges, competition-level problems, and undergraduate STEM tasks, including *GSM-Plus* [48], *OlympiadBench* [49], *AIME I/II* (2024–2025), *Minerva* [50], and *MATH-500* [41]. Across these benchmarks, we evaluate models along two complementary dimensions: *answer accuracy* and *token usage*. Performance is measured using sampling-based *pass@k* [51], which reflects the probability that at least one of k sampled outputs is correct. Efficiency is quantified by the average number of generated tokens per instance, computed independently of the *pass@k* sampling budget. Final answers are validated using a dual-verifier strategy: predictions are deemed correct if accepted by either *Math-Verify* [52] or *MARIO Eval* [53], which together provide format-agnostic and type-aware symbolic and numerical equivalence checking. Further details of the evaluation settings are provided in § B.

5.3 Baselines

We evaluate *When2Think* against representative baselines from three categories: (1) base LLMs, (2) LRMs trained via SFT or RFT, and (3) efficiency-aware LRMs.

We include both general-purpose and math-specialized LLMs, including *Qwen2.5-Instruct* [54] and *Qwen2.5-Math-Instruct* [44]. For explicit reasoning, we evaluate *R1-Distill-Qwen* [32], which also serves as the backbone of *When2Think*, and *DeepScaleR-Preview* [45], an trained on the same *DeepScaleR* dataset. We compare *When2Think* against representative efficiency-aware reasoning models. Length-compression baselines, including *LC-R1* [22], *ThinkPrune* [23], and *LASER* [24], primarily reduce computation via global or heuristic length constraints. Hybrid reasoning baselines, such as *AdaptThink* [28] and *ThinkLess* [29], instead reduce cost by discretely skipping explicit reasoning on easier instances.

All methods share the same base model and are trained under comparable conditions, ensuring that performance differences reflect the impact of adaptive reasoning rather than variations in model or supervision. For baselines that use the same training dataset [24, 28, 29], we follow their original data setup. All baseline models are implemented using open weights.

6 Experimental Results & Analysis

We evaluate *When2Think* through three questions covering trade-off, allocation, and mechanism.

RQ1: Accuracy–Efficiency Trade-off under Adaptive Allocation. Does *When2Think* improve the accuracy–efficiency trade-off and mitigate the *efficiency tax*?

RQ2: Difficulty-aware Adaptive Allocation between System 1 and 2. Does the policy allocate less computation to easy instances and preserve deeper reasoning on hard ones?

RQ3: Mechanisms Effects. How do *Instance-level Difficulty-Aware Control (IDAC)*, *Batch-Wise Standardization (BWS)*, *Importance Sampling (IS)* enable stable control of reasoning depth control?

Table 1: **Performance comparison on mathematical reasoning benchmarks.** We report Pass@3 accuracy (% , higher is better), average tokens per response (Tokens, lower is better) on *GSM-Plus*, *OlympiadBench-Math*, and *AIME24*. All results are obtained using sampling-based Pass@k with matched decoding settings and are reported as mean $\pm$ standard deviation over five independent runs. *When2Think* is evaluated alongside base LLMs, LRM-based baselines, and efficiency-oriented reasoning models. Values in parentheses indicate differences relative to R1-Distill-Qwen (**green**: improvement; **red**: degradation).

Model \ Benchmark	GSM-Plus		OlympiadBench-Math		AIME24		AIME25	
	↑ Pass@3(%)	↓ Tokens(#)	↑ Pass@3(%)	↓ Tokens(#)	↑ Pass@3(%)	↓ Tokens(#)	↑ Pass@3(%)	↓ Tokens(#)
Qwen2.5-Math-Instruct	84.3 $\pm$ 0.3	360 $\pm$ 1.8	53.2 $\pm$ 0.8	842 $\pm$ 8.3	22.7 $\pm$ 1.5	1,057 $\pm$ 57.7	19.3 $\pm$ 2.8	923 $\pm$ 64.8
R1-Distill-Qwen (Base)	79.4 $\pm$ 0.3	590 $\pm$ 11.6	58.8 $\pm$ 2.0	8,634 $\pm$ 107.4	46.0 $\pm$ 2.8	14,195 $\pm$ 1,026.7	32.0 $\pm$ 5.1	12,616 $\pm$ 688.4
DeepScaleR-Preview	85.4 $\pm$ 0.3	1,358 $\pm$ 13.4	62.8 $\pm$ 0.8	5,215 $\pm$ 43.3	58.0 $\pm$ 5.6	8,473 $\pm$ 479.5	39.3 $\pm$ 6.8	8,074 $\pm$ 288.6
LC-R1	79.0 $\pm$ 0.3 (- 0.4)	546 $\pm$ 9.9 (- 44)	55.5 $\pm$ 1.1 (- 3.3)	4,092 $\pm$ 113.0 (- 4,542)	36.0 $\pm$ 4.9 (- 10.0)	7,972 $\pm$ 1,024.3 (- 6,223)	32.0 $\pm$ 3.8 ($\pm$ 0)	6,971 $\pm$ 278.7 (- 5,645)
ThinkPrune-iter3k	81.8 $\pm$ 0.3 (+ 2.4)	609 $\pm$ 5.5 (+ 19)	58.6 $\pm$ 0.7 (+ 0.2)	3,781 $\pm$ 66.5 (- 4,853)	46.0 $\pm$ 4.3 ($\pm$ 0)	6,236 $\pm$ 555.4 (- 7,959)	36.0 $\pm$ 3.7 (+ 4.0)	5,582 $\pm$ 273.3 (- 7,034)
Laser-DE-L4096	80.1 $\pm$ 0.2 (+ 0.7)	562 $\pm$ 4.1 (- 28)	59.9 $\pm$ 1.1 (+ 1.1)	4,830 $\pm$ 43.4 (- 3,804)	45.3 $\pm$ 3.8 (- 0.7)	8,425 $\pm$ 362.3 (- 5,770)	37.3 $\pm$ 8.9 (+ 5.3)	6,971 $\pm$ 1,024.3 (- 5,645)
AdaptThink-delta0.05	83.6 $\pm$ 0.2 (+ 3.2)	716 $\pm$ 17.5 (+ 126)	58.4 $\pm$ 0.8 (- 0.4)	3,004 $\pm$ 76.2 (- 5,630)	44.7 $\pm$ 4.5 (- 1.3)	5,806 $\pm$ 438.8 (- 8,389)	30.7 $\pm$ 8.0 (- 1.3)	6,883 $\pm$ 440.2 (- 5,733)
Thinkless-RL	85.8 $\pm$ 0.2 (+ 6.4)	1,799 $\pm$ 16.2 (+ 1,209)	63.4 $\pm$ 1.1 (+ 4.6)	6,960 $\pm$ 76.2 (- 1,674)	46.7 $\pm$ 4.1 (+ 0.7)	11,023 $\pm$ 209.4 (- 3,172)	33.3 $\pm$ 7.5 (+ 1.3)	11,056 $\pm$ 641.7 (- 1,560)
When2Think (Ours)	85.7 $\pm$ 0.3 (+ 6.3)	1,052 $\pm$ 17.9 (+ 462)	62.4 $\pm$ 0.9 (+ 6.3)	5,909 $\pm$ 56.4 (- 2,725)	56.0 $\pm$ 2.8 (+ 10.0)	10,236 $\pm$ 438.0 (- 3,959)	40.0 $\pm$ 0.0 (+ 8.0)	9,549 $\pm$ 719.4 (- 3,067)
w/o IS (IDAC + BWS)	86.8 $\pm$ 0.1 (+ 7.4)	1,652 $\pm$ 13.2 (+ 1,062)	65.9 $\pm$ 1.2 (+ 7.1)	6,239 $\pm$ 87.8 (- 2,395)	57.3 $\pm$ 8.3 (+ 11.3)	10,046 $\pm$ 487.9 (- 4,149)	40.0 $\pm$ 3.3 (+ 8.0)	9,846 $\pm$ 269.2 (- 2,770)

6.1 RQ1: Accuracy–Efficiency Trade-off under Adaptive Allocation

We evaluate whether *When2Think* improves the accuracy–efficiency trade-off by reducing unnecessary reasoning on easy instances without inducing under-allocation on hard ones.

The Efficiency Tax. Many efficient reasoning methods reduce computation through fixed length constraints, heuristic compression, or discrete mode switching. While these strategies can suppress redundant reasoning on easy instances, they may incur an *efficiency tax*: token savings are obtained by removing reasoning that is still needed on harder instances. For example, on AIME24 (Table 1), *LC-R1* and *AdaptThink* reduce inference cost relative to the *R1-Distill-Qwen* backbone, but their accuracy decreases by 10.0 and 1.3 points, respectively. A similar hard-instance under-allocation pattern appears on MATH-500 Level 5 (Table 10 in § C.3).

Top-Right Accuracy–Efficiency Improvements. Figure 6 shows the accuracy–efficiency landscape across five benchmarks: GSM-Plus, Minerva, OlympiadBench, AIME24, and AIME25. Since the token axis is reversed, movement toward the top-right indicates a better trade-off: higher accuracy with fewer tokens. Our claim is not that *When2Think* is always the lowest-token operating point; routing-only methods such as *AdaptThink* can be strong low-cost points on easier tasks. Rather, *When2Think* aims to reduce unnecessary computation without converting those savings into hard-instance accuracy loss. On AIME24, it reduces token usage by 3,959 tokens (-27.9%) while improving accuracy from 46.0% to 56.0% (10.0%), showing that adaptive allocation can mitigate the efficiency tax rather than merely shift it.

6.2 RQ2: Difficulty-aware Adaptive Allocation between System 1 and 2

We analyze whether *When2Think* dynamically allocates computation in a *difficulty-aware* manner, balancing SYSTEM 1/2 across benchmarks of varying difficulty.

SYSTEM 1 Preference on Easy Instances. On easy MATH-500 (Table 10 in § C.3) Level 1 problems, the *R1-Distill* generates an average of 1,199 tokens despite minimal reasoning demand. In contrast, *When2Think* reduces token usage to 619 (-580) while maintaining high accuracy (95.8%), indicating a clear preference for SYSTEM 1-style direct answering when additional deliberation provides limited benefit. This pattern generalizes to easy GSM-Plus (Table 1) problems.

SYSTEM 2 Engagement on Hard Instances. On hard MATH-500 (Table 10 in § C.3) Level 5 problems, *When2Think* maintains accuracy while reducing token by 2,276 relative to *R1-Distill*,

indicating more selective yet sustained SYSTEM 2 reasoning. In contrast, on adversarially perturbed GSM-Plus instances, it deliberately *increases* computation (462 tokens), yielding a 6.3% absolute accuracy gain. On long-horizon tasks such as AIME24/25 (Table 1), *When2Think* maintains sufficient deliberation depth, while compression-based and discrete hybrid methods often truncate reasoning prematurely, leading to poorer performance on hard instances.

Instance-Level Difficulty-Aware Allocation. *When2Think* adapts reasoning behavior at the instance level. On MATH-500 (Figure 4 and § C.2), the fraction of THINK trajectories increases monotonically with difficulty, from about 0.2 at Level 1 to over 0.7 at Level 5. By contrast, baselines exhibit weak or misaligned responses: *ThinkLess* allocates nearly constant computation, *DeepScaleR* overthinks easy instances, and *AdaptThink* under-allocates on hard ones. This demonstrates that *When2Think* more accurately matches computation to problem difficulty.

6.3 RQ3: Mechanism Effects

We analyze the mechanisms underlying *When2Think*, focusing on the roles of *Instance-level Difficulty-Aware Control (IDAC)*, *Batch-Wise Standardization (BWS)*, *Importance Sampling (IS)*, and model scale. Additional ablations are provided in § D.

IDAC enables depth control. Compared with *AdaptThink* [28], which primarily learns whether to invoke THINK, *When2Think* further controls how deeply to reason once THINK is selected. On AIME24 (Table 1), *AdaptThink* uses fewer tokens (5,806) but drops to 44.7% Pass@3, below the *R1-Distill-Qwen* backbone, whereas *When2Think* achieves 56.0% Pass@3 while still reducing token usage. Consistent with this, the *IS+BWS* variant—which retains balanced THINK/NOTHINK exploration but removes IDAC—exhibits substantially worse validation accuracy, particularly on the hard AIME24 benchmark (Figure 9 and § D.3).

BWS stabilizes optimization. BWS converts IDAC-shaped trajectory rewards into standardized advantages, enabling stable critic-free optimization. Beyond variance reduction, it preserves cross-instance difficulty structure, allowing successful long trajectories on hard instances to receive positive credit. As shown in §§ D.1 and D.2, BWS improves convergence, stabilizes response length and think ratio, and prevents degenerate reasoning behaviors, maintaining a stable think ratio of approximately 0.9 compared to 0.6 without BWS (Figure 8).

IS improves efficiency, not accuracy. Removing importance sampling (while retaining IDAC and BWS, Table 1) yields a strong variant that outperforms *DeepScaleR-Preview* on most benchmarks, indicating that the primary accuracy gains stem from IDAC and BWS. However, IS shifts the policy toward a more efficient operating point: compared with the w/o IS variant, the full *When2Think* model substantially reduces GSM-Plus token usage (1,652 $\rightarrow$ 1,052 tokens) while maintaining competitive accuracy. This suggests that IS primarily promotes effective NOTHINK usage on easier instances, rather than serving as the main source of accuracy gains (§ D.3).

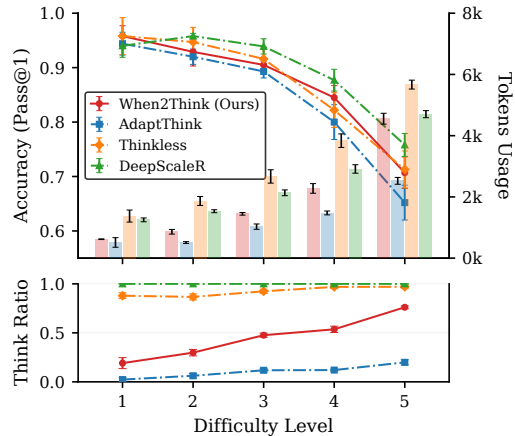


Figure 4: **Performance and behavioral analysis on the MATH-500 benchmark** stratified by problem difficulty (Levels 1–5). Results are reported for *When2Think* (red), *DeepScaleR* (green), *Thinkless* (orange), and *AdaptThink* (blue). **(Top) Accuracy and efficiency:** Pass@1 accuracy (lines) and average token usage (bars). **(Bottom) Think ratio:** Fraction of trajectories that trigger explicit reasoning across difficulty levels. All results are reported as mean $\pm$ standard deviation over five independent runs.

7 Discussion

7.1 The Efficiency Tax: From Compression to Control

We argue that reasoning efficiency in LRMs is better understood as an *instance-adaptive control problem*, rather than a pure *compression problem*. Prior approaches reduce computation using static or heuristic length constraints, which suppress overthinking on easy instances but induce an *efficiency tax* by under-allocating reasoning on harder problems. This limitation suggests that inefficiency arises from misallocation of computation rather than insufficient model capacity. The strong performance of DeepScaleR-Preview further indicates that efficiency gains can emerge from improved allocation across instances, rather than compression alone.

From Compression to Control. *When2Think* treats reasoning depth as an instance-adaptive resource, reallocating computation by suppressing redundant reasoning on easy problems while preserving or expanding deliberation on hard ones. This shifts the objective from minimizing reasoning length to optimizing where and how much reasoning is applied.

7.2 Mechanisms: Continuous Control and Stability

IDAC is Necessary for Depth Allocation. While hybrid reasoning methods can learn when to invoke THINK, our results show that mode selection alone is insufficient. Removing IDAC while retaining balanced exploration and stable optimization leads to a substantial drop in accuracy, particularly on hard benchmarks such as AIME24 (Figure 9 and § D.3). Although such variants can still decide when to reason, they lack a mechanism for determining how much reasoning to allocate, resulting in systematic under-allocation on difficult instances. These findings establish that difficulty-aware depth control is essential beyond discrete routing. *When2Think* implements this depth control through continuous modulation rather than discrete decisions. IDAC leverages instance-level statistics (α_i, τ_i) to scale reasoning incentives, forming a spectrum from direct answers to extended deliberation. This avoids abrupt truncation and enables instance-aware allocation of reasoning depth.

Stable Optimization via BWS. Learning instance-adaptive control is challenging due to difficulty-dependent reward variability. *When2Think* addresses this with two mechanisms: instance-specific baselines α_i , which eliminate the need for a learned critic, and *Batch-Wise Standardization (BWS)*, which normalizes advantages across the mini-batch. This decouples difficulty from optimization signals and enables stable, critic-free learning of length-sensitive policies.

7.3 The Paradox of Scale: Overthinking and the Visibility of Efficiency Tax

Smaller reasoning models tend to generate longer but less effective reasoning traces, leading to systematic overthinking (§ F). As a result, inefficiencies such as the efficiency tax are most visible at smaller scales (Tables 9 and 10). Although *When2Think* generalizes to larger models—maintaining strong performance on hard problems while reducing unnecessary computation—larger models exhibit stronger inherent reasoning efficiency, making these effects substantially less pronounced. Accordingly, we focus our analysis on the 1.5B scale, where inefficient computation is most diagnostic and where adaptive control mechanisms such as IDAC have the greatest impact.

7.4 Limitations

Our approach relies on verifiable rewards, and is thus most directly applicable to domains such as mathematical reasoning. Extending *When2Think* to open-ended tasks requires additional reward modeling or human evaluation. Moreover, our method depends on offline reference statistics for difficulty estimation, which may be less reliable when problem difficulty is ambiguous.

8 Conclusion

We argue that inefficiency in large reasoning models arises not from excessive reasoning, but from an *efficiency tax* caused by misallocated reasoning depth across instances. Existing approaches reduce

computation via static compression, but often do so at the cost of under-allocating reasoning on difficult problems. We propose *When2Think*, which treats reasoning depth as an instance-adaptive resource rather than a quantity to be minimized. Through *Instance-level Difficulty-Aware Control (IDAC)*, the model learns not only when to reason but how much to reason. Combined with *Batch-Wise Standardization (BWS)*, this enables stable, critic-free optimization of depth-aware policies. Across diverse mathematical benchmarks, *When2Think* improves the accuracy–efficiency trade-off by reducing unnecessary computation while preserving or expanding reasoning where it is needed. Overall, our results suggest that efficient reasoning is fundamentally a problem of *allocation rather than compression*, providing a principled direction for scalable reasoning systems.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, et al. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- [2] Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, et al. Scaling language models: Methods, analysis & insights from training gopher, 2022. URL <https://arxiv.org/abs/2112.11446>.
- [3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, et al. An empirical analysis of compute-optimal large language model training. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf.
- [4] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24 (240):1–113, 2023. URL <http://jmlr.org/papers/v24/22-1144.html>.
- [5] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, et al. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [6] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 15476–15488. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/639a9a172c044fbb64175b5fad42e9a5-Paper-Conference.pdf.
- [7] Zayne Rea Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To cot or not to cot? chain-of-thought helps mainly on math and symbolic reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=w6nlcS8Kkn>.
- [8] Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A survey. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1049–1065, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.67. URL <https://aclanthology.org/2023.findings-acl.67/>.
- [9] Guiyao Tie, Zeli Zhao, Dingjie Song, Fuyang Wei, Rong Zhou, et al. A survey on post-training of large language models, 2025. URL <https://arxiv.org/abs/2503.06072>.
- [10] Komal Kumar, Tajamul Ashraf, Omkar Thawakar, Rao Muhammad Anwer, Hisham Cholakkal, et al. Llm post-training: A deep dive into reasoning large language models, 2025. URL <https://arxiv.org/abs/2502.21321>.

- [11] Zixuan Ke, Fangkai Jiao, Yifei Ming, Xuan-Phi Nguyen, Austin Xu, et al. A survey of frontiers in LLM reasoning: Inference scaling, learning to reason, and agentic systems. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=SlsZZ25InC>. Survey Certification.
- [12] Fengli Xu, Qianye Hao, Chenyang Shao, Zefang Zong, Yu Li, et al. Toward large reasoning models: A survey of reinforced reasoning with large language models. *Patterns*, 6(10):101370, 2025. ISSN 2666-3899. doi: <https://doi.org/10.1016/j.patter.2025.101370>. URL <https://www.sciencedirect.com/science/article/pii/S2666389925002181>.
- [13] Duzhen Zhang, Zhong-Zhi Li, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, et al. From system 1 to system 2: A survey of reasoning large language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pages 1–20, 2025. doi: 10.1109/TPAMI.2025.3637037.
- [14] Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. Efficient reasoning models: A survey. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=sySqlxj8EB>.
- [15] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, et al. Do NOT think that much for $2+3=?$ on the overthinking of long reasoning models. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=MSbU3L7V00>.
- [16] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, et al. Stop overthinking: A survey on efficient reasoning for large language models. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=HvoG8SxggZ>.
- [17] Yue Wang, Qiuzhi Liu, Jiahao Xu, Tian Liang, Xingyu Chen, et al. Thoughts are all over the place: On the underthinking of long reasoning models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=WcUo7Z2Jnh>.
- [18] Steven A. Sloman. The empirical case for two systems of reasoning. *Psychological Bulletin*, 119(1):3–22, 1996. doi: 10.1037/0033-2909.119.1.3.
- [19] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, USA, 2004. ISBN 052154310X.
- [20] Parshin Shojaee, Seyed Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=YghiOusmvw>.
- [21] Xiaoshu Chen, Sihang Zhou, Ke Liang, Xiaoyu Sun, and Xinwang Liu. Skip-thinking: Chunk-wise chain-of-thought distillation enable smaller language models to reason better and faster. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 12153–12168, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.610. URL <https://aclanthology.org/2025.emnlp-main.610/>.
- [22] Zhengxiang Cheng, Dongping Chen, Mingyang Fu, and Tianyi Zhou. Optimizing length compression in large reasoning models, 2025. URL <https://arxiv.org/abs/2506.14755>.
- [23] Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, et al. Thinkprune: Pruning long chain-of-thought of llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.01296>.

- [24] Wei Liu, Ruochen Zhou, Yiyun Deng, Yuzhen Huang, Junteng Liu, Yuntian Deng, Yizhe Zhang, and Junxian He. Learn to reason efficiently with adaptive length-based reward shaping. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=hj9eKpqxQL>.
- [25] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Mingjie Liu, et al. Dler: Doing length penalty right - incentivizing more intelligence per token via reinforcement learning, 2025. URL <https://arxiv.org/abs/2510.15110>.
- [26] Siye Wu, Jian Xie, Yikai Zhang, Aili Chen, Kai Zhang, et al. ARM: Adaptive reasoning model. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL <https://openreview.net/forum?id=FCcQ7o0WdS>.
- [27] Lingjie Jiang, Xun Wu, Shaohan Huang, Qingxiu Dong, Zewen Chi, et al. Think only when you need with large hybrid-reasoning models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=fDjDVE4qdj>.
- [28] Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. AdaptThink: Reasoning models can learn when to think. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3716–3730, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.184. URL <https://aclanthology.org/2025.emnlp-main.184/>.
- [29] Gongfan Fang, Xinyin Ma, and Xinchao Wang. Thinkless: LLM learns when to think. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=arivQfOKZx>.
- [30] OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, et al. Openai o1 system card, 2024. URL <https://arxiv.org/abs/2412.16720>.
- [31] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025. URL <https://qwenlm.github.io/blog/qwq-32b/>.
- [32] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- [33] Marah Abidin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, et al. Phi-4-reasoning technical report, 2025. URL <https://arxiv.org/abs/2504.21318>.
- [34] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025. URL <https://arxiv.org/abs/2507.06261>.
- [35] Maciej Besta, Julia Barth, Eric Schreiber, Ales Kubicek, Afonso Catarino, et al. Reasoning language models: A blueprint, 2025. URL <https://arxiv.org/abs/2501.11223>.
- [36] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, et al. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf.
- [37] Luong Trung, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. ReFT: Reasoning with reinforced fine-tuning. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7601–7614, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.410. URL <https://aclanthology.org/2024.acl-long.410/>.

- [38] Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, et al. A survey of reinforcement learning for large reasoning models, 2025. URL <https://arxiv.org/abs/2509.08827>.
- [39] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, et al. Tulu 3: Pushing frontiers in open language model post-training. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=i1uGbfHHpH>.
- [40] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, et al. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [41] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, et al. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v8L0pN6E0i>.
- [42] Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking, 2025. URL <https://arxiv.org/abs/2504.09858>.
- [43] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. Reinforce++: Stabilizing critic-free policy optimization with global advantage normalization, 2025. URL <https://arxiv.org/abs/2501.03262>.
- [44] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, et al. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024. URL <https://arxiv.org/abs/2409.12122>.
- [45] Sijun Tan, Michael Luo, Justin Wong, Colin Cai, Xiaoxiang Shi, William Yuan Tang, Manan Roongta, Tianjun Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Effective RL scaling of reasoning models via iterative context lengthening, 2026. URL <https://openreview.net/forum?id=I6GzDCne7U>.
- [46] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, et al. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=yaqPf0KA1N>.
- [47] Yingqian Min, Zhipeng Chen, Jinhao Jiang, Jie Chen, Jia Deng, et al. Imitate, explore, and self-improve: A reproduction report on slow-thinking reasoning systems, 2024. URL <https://arxiv.org/abs/2412.09413>.
- [48] Qintong Li, Leyang Cui, Xueliang Zhao, Lingpeng Kong, and Wei Bi. GSM-plus: A comprehensive benchmark for evaluating the robustness of LLMs as mathematical problem solvers. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2961–2984, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.163. URL <https://aclanthology.org/2024.acl-long.163/>.
- [49] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, et al. OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.211. URL <https://aclanthology.org/2024.acl-long.211/>.
- [50] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, et al. Solving quantitative reasoning problems with language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 3843–3857. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/18abbef8cfe9203fdf9053c9c4fe191-Paper-Conference.pdf.

- [51] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [52] Hynek Kydlíček. Math-Verify: Math Verification Library. URL <https://github.com/huggingface/math-verify>.
- [53] Boning Zhang, Chengxi Li, and Kai Fan. Mario eval: Evaluate your math llm with your math llm—a mathematical dataset evaluation toolkit, 2024. URL <https://arxiv.org/abs/2404.13925>.
- [54] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>.
- [55] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, et al. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’25, page 1279–1297, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711961. doi: 10.1145/3689031.3696075. URL <https://doi.org/10.1145/3689031.3696075>.
- [56] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP ’23, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- [57] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, et al. Measuring mathematical problem solving with the math dataset. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/be83ab3ecd0db773eb2dc1b0a17836a1-Paper-round2.pdf.
- [58] Ximing Lu, Seungju Han, David Acuna, Hyunwoo Kim, Jaehun Jung, et al. Retro-search: Exploring untaken paths for deeper and efficient reasoning, 2025. URL <https://arxiv.org/abs/2504.04383>.
- [59] Zhiyuan Li, Yi Chang, and Yuan Wu. Think-bench: Evaluating thinking efficiency and chain-of-thought quality of large reasoning models, 2025. URL <https://arxiv.org/abs/2505.22113>.
- [60] Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, et al. Olmo 3, 2025. URL <https://arxiv.org/abs/2512.13961>.
- [61] Magda Osman. An evaluation of dual-process theories of reasoning. *Psychonomic Bulletin & Review*, 11(6):988–1010, December 2004.
- [62] Sean Welleck, Amanda Bertsch, Matthew Finlayson, Hailey Schoelkopf, Alex Xie, et al. From decoding to meta-generation: Inference-time algorithms for large language models. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=eskQMCIbMS>. Survey Certification.
- [63] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, et al. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.
- [64] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, et al. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2609–2634,

- Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.147. URL <https://aclanthology.org/2023.acl-long.147/>.
- [65] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, et al. Self-refine: Iterative refinement with self-feedback. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf.
 - [66] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, et al. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
 - [67] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, et al. Tree of thoughts: Deliberate problem solving with large language models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/271db9922b8d1f4dd7aaef84ed5ac703-Paper-Conference.pdf.
 - [68] Kanishk Gandhi, Denise H J Lee, Gabriel Grand, Muxin Liu, Winson Cheng, et al. Stream of search (sos): Learning to search in language. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=2cop2jmQVL>.
 - [69] Jiayi Pan, Xiuyu Li, Long Lian, Charlie Victor Snell, Yifei Zhou, et al. Learning adaptive parallel reasoning with language models. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=YgwQ7sXPXU>.
 - [70] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024. URL <http://jmlr.org/papers/v25/23-0870.html>.
 - [71] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. URL <https://arxiv.org/abs/1503.02531>.
 - [72] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In Jian Su, Kevin Duh, and Xavier Carreras, editors, *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1317–1327, Austin, Texas, November 2016. Association for Computational Linguistics. doi: 10.18653/v1/D16-1139. URL <https://aclanthology.org/D16-1139/>.
 - [73] Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14852–14882, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.830. URL <https://aclanthology.org/2023.acl-long.830/>.
 - [74] Dan Zhang, Sining Zhoubian, Ziniu Hu, Yisong Yue, Yuxiao Dong, and Jie Tang. Rest-mcts*: Llm self-training via process reward guided tree search. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 64735–64772. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/76ec4dc30e9faaf0e4b6093eaa377218-Paper-Conference.pdf.
 - [75] Yuxi Xie, Anirudh Goyal, Wenye Zheng, Min-Yen Kan, Timothy P Lillicrap, et al. Monte carlo tree search boosts reasoning via iterative preference learning. In *The First Workshop on System-2 Reasoning at Scale, NeurIPS’24*, 2024. URL <https://openreview.net/forum?id=s0040mYP2P>.

- [76] Hyeonbin Hwang, Doyoung Kim, Seungone Kim, Seonghyeon Ye, and Minjoon Seo. Self-explore: Enhancing mathematical reasoning in language models with fine-grained rewards. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1444–1466, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.78. URL <https://aclanthology.org/2024.findings-emnlp.78/>.

A Implementation details

A.1 Reproducibility

All experiments prioritize transparency and reproducibility. To ensure deterministic behavior, the random seed was fixed to 42 across all processes, including data shuffling, model initialization, and sampling. Hyperparameter configurations (Table 2) follow prior studies on reasoning alignment for fair comparison.

Table 2: **Hyperparameter** for training configuration.

Hyperparameter	1.5B	7B
<i>Optimization</i>		
Total Episodes	40,315	40,315
Total Epochs	1	1
Total Steps	314	628
Batch Size	128	64
Learning Rate	3×10^{-6}	7×10^{-7}
Dynamic Batch	True	False
<i>Generation & Architecture</i>		
Max Prompt Length	1,024	1,024
Max THINK Response Length	16,384	16,384
Max NOTHINK Response Length	4,096	4,096
Reference Sampling (K)	16	16
Importance Sampling (K)	16	16
Temperature	0.6	0.6
Top-p	0.95	0.95
Random Seed	42	42
Weight Type	float32	float32
<i>Reward Configuration (When2Think)</i>		
Reference Scaler (γ)	0.9	0.9
NOTHINK Ratio	0.5	0.5
Bonus (δ)	0.05	0.05

A.2 Infrastructure

Training the 1.5B model required approximately 70 GPU-hours on $2 \times$ NVIDIA H100 80GB GPUs, while the 7B model required approximately 120 GPU-hours. Evaluation across all reported benchmarks required approximately 20 GPU-hours on $2 \times$ NVIDIA A100 80GB GPUs. Training throughput was maximized using the *verl* framework [55] with Distributed Data Parallel (DDP) and FlashAttention-2 integration. Inference and evaluation employed the *vLLM* engine [56], which reduces memory overhead and latency for long chain-of-thought generation. Comprehensive software and hardware specifications are summarized in Table 3.

Table 3: **Computational environment and software stack** used in experiments.

Environment	Train	Test
<i>Hardware Accelerator</i>		
Accelerator	NVIDIA H100 (80GB HBM3)	NVIDIA A100 (80GB)
<i>System Environment</i>		
OS Kernel	Linux-5.15.0-161-generic	Linux-5.4.0-173-generic
Python	CPython-3.11.14	CPython 3.11.14
CUDA	13.0	12.8
<i>Core Frameworks</i>		
Frameworks	PyTorch-2.9.0	PyTorch-2.9.0
	Transformers-4.57.1	Transformers-4.57.3
	verl-0.2.0.dev	vLLM-0.11.2

B Evaluation Setup

B.1 Inference and Verification Protocol

To ensure a rigorous and reproducible evaluation, we standardize inference-time generation parameters and the answer verification pipeline across all models.

Generation Configuration. We adopt decoding strategies tailored to the characteristics of each model class while keeping them fixed within each category for fair comparison. For general-purpose LLMs, we use a `temperature` of 0.7 and `top-p` of 0.8 to encourage diverse solution exploration. For Large Reasoning Models (LRMs), we apply a more deterministic configuration with a `temperature` of 0.6 and `top-p` of 0.95, which stabilizes long-horizon reasoning while preserving sufficient diversity in intermediate steps. This distinction follows common practice in prior work and accounts for the differing sensitivity of LLMs and LRMs to sampling noise during reasoning, while all generation settings are held constant across benchmarks to ensure fair comparison.

To ensure reliable answer extraction and automated evaluation, all models are instructed to follow a strict output format using the system prompt shown in § B.1, which is instantiated using each model’s native chat template.

System Prompt for Mathematical Reasoning

Math assistant. Provide a step-by-step solution in English using $\LaTeX$. Final answer in `\boxed{\dots}`.

Answer Extraction and Verification. We report `Pass@k` accuracy based on answers extracted from the final `\boxed{\dots}` tag. To ensure robustness to formatting variations and symbolic ambiguity, we employ a **Dual Math Verifier** pipeline that integrates *Math-Verify* [52] and *MARIO Eval* [53]. Math-Verify performs primary extraction and symbolic equivalence checking, while MARIO Eval provides complementary semantic verification using type-specific Computer Algebra Systems (CAS). The resulting unified binary correctness signal is used consistently as both the evaluation metric and the terminal reward for policy optimization.

B.2 Dual Math Verifier

Accurate and robust verification is essential for evaluating mathematical reasoning and for constructing reliable reward signals during policy optimization. Because mathematically correct answers can be expressed in multiple symbolic or numerical forms, exact string matching is often insufficient. We therefore employ a dual-verifier pipeline that evaluates semantic equivalence between model-generated answers and ground-truth solutions.

Math-Verify. We use Math-Verify [52] as a format-agnostic verifier for mathematical reasoning tasks. Given a model-generated response, Math-Verify extracts the candidate answer using prioritized parsing rules that support diverse output formats, including $\LaTeX$ expressions and plain text. The extracted answer is then normalized and converted into a symbolic representation, enabling comparison with the ground-truth solution beyond exact string matching. Verification is performed through symbolic equivalence checking and numerical evaluation with tolerance, allowing equivalent mathematical expressions to be recognized even when their surface forms differ.

MARIO Eval. We additionally adopt MARIO Eval [53] to provide type-aware verification across diverse mathematical answer formats. MARIO Eval first identifies the answer type, such as a scalar, algebraic expression, matrix, equation, or inequality, using rule-based parsing and optional language-model assistance when type inference is ambiguous. It then applies type-specific equivalence checks, primarily relying on symbolic computation through a computer algebra system and numerical evaluation with tolerance. When symbolic verification is inconclusive due to structural ambiguity or natural-language formulations, MARIO Eval can optionally invoke a math-capable language model as a fallback judge.

B.3 Benchmarks

We evaluate models on a diverse suite of mathematical reasoning benchmarks spanning foundational arithmetic, intermediate competition problems, olympiad-level challenges, and undergraduate STEM tasks. Together, these benchmarks cover a broad difficulty spectrum, enabling fine-grained analysis of both reasoning accuracy and computational efficiency across problem complexities.

GSM-Plus. GSM-Plus [48] is a medium-scale benchmark of approximately 10.5k problems that extends GSM-8K [40] with systematically perturbed variants. It introduces controlled modifications, including paraphrasing, numerical changes, distractor insertion, and reasoning-structure perturbations, enabling evaluation of robustness under increased reasoning complexity and distribution shift. Although GSM-Plus contains a small number of instances annotated with None answers for unanswerable or incomplete cases, we exclude them to maintain a consistent answer-extraction and verification protocol. This filtering yields a final evaluation set of approximately 9k problems.

MATH-500. The MATH-500 benchmark [41] is a held-out evaluation subset derived from the original MATH [57] dataset, preserving its structure while avoiding contamination from the PRM-800K [41] training pipeline. The full MATH dataset spans seven subjects—Prealgebra, Algebra, Number Theory, Counting and Probability, Geometry, Intermediate Algebra, and Precalculus—and assigns each problem a difficulty level from 1 to 5 based on AoPS conventions, ranging from introductory AMC 8-style problems to AIME-level items. To prevent overfitting due to inclusion of test-split MATH problems in PRM-800K, 500 uniformly sampled problems were isolated to form MATH-500. This subset retains the subject distribution and difficulty profile of the original test split, providing a leakage-free benchmark for evaluating advanced mathematical reasoning.

Minerva. The Undergraduate-Level STEM Problems, also known as the Minerva benchmark and referred to as OCW Courses in Lewkowycz et al. [50], consists of 272 self-contained problems collected from MIT OpenCourseWare materials across multiple STEM subjects, including Solid-State Chemistry, Information and Entropy, Differential Equations, and Special Relativity. Each problem provides a clearly delineated final answer and is automatically verifiable, either numerically (191 problems) or symbolically via SymPy (81 problems), emphasizing multi-step reasoning while excluding formal proofs or open-ended short answers. This benchmark is specifically designed to evaluate scientific and quantitative reasoning capabilities in large language models at the undergraduate level, bridging the gap between standard K-12 mathematics benchmarks and more advanced domain-specific problem-solving tasks.

OlympiadBench-Math. OlympiadBench [49] is a high-difficulty, bilingual benchmark comprising 8,476 problems across multiple STEM domains in both multimodal and text-only formats. For evaluation of mathematical reasoning, only text-only math problems are used to isolate multi-step reasoning performance. Each problem includes an expert-verified solution, enabling reliable assessment of model reasoning. Experiments are conducted on a curated subset of 674 open-ended, text-only English problems from the Math Competition track of OlympiadBench.

AIME. The American Invitational Mathematics Examination (AIME) is a selective competition featuring integer-answer problems ranging from 000 to 999. These problems are substantially more challenging than standard AMC questions, requiring multi-step reasoning, careful case analysis, and deeper understanding of algebra, combinatorics, number theory, and geometry. For evaluation, both administrations (AIME I and AIME II) are combined to form a set of 30 problems. The 2024 and 2025 edition is used to avoid overlap with the DeepScaleR dataset, which includes material only through 2023.

B.4 Thoughts & Steps

Lu et al. [58] propose a hierarchical decomposition of reasoning trajectories by distinguishing between *thoughts* and *steps*.

Given a question $q \in \mathcal{Q}$, a reasoning model $\mathcal{M}$ produces a reasoning trajectory T and a final answer a , denoted as $(T, a) := \mathcal{M}(q)$. The trajectory T is organized as a sequence of higher-level thoughts, each of which consists of multiple intermediate reasoning steps.

Thoughts (s^τ). A reasoning trajectory T is represented as an ordered sequence of thoughts:

$$T := \langle s^1, s^2, \dots, s^\tau, \dots, s^m \rangle, \quad (9)$$

where each s^τ corresponds to a contiguous segment of reasoning that follows a coherent line of argument or problem-solving strategy. Transitions between consecutive thoughts, $s^\tau \rightarrow s^{\tau+1}$, are identified using discourse-level linguistic markers (e.g., “Alternatively”, “Wait”; see § B.4), which heuristically indicate a shift in reasoning direction.

Discourse Markers Used for Segmenting Thoughts

But, Wait, Alternatively, However, Hmm, Hmmm, Not sure, Going back, Backtrack, Trace back, Another

Steps (s_k^τ). Each thought s^τ is further decomposed into a sequence of intermediate steps:

$$s^\tau := \langle s_1^\tau, s_2^\tau, \dots, s_{k_\tau}^\tau \rangle. \quad (10)$$

The superscript τ indexes the thought, while the subscript k indexes the step within that thought. In practice, steps correspond to fine-grained inference units and are delimited using formatting cues in the generated trace, such as double newline characters (`\n\n`).

Under this formulation, a reasoning trajectory admits the following hierarchical structure:

$$T = \langle \langle s_1^1, \dots, s_{k_1}^1 \rangle, \dots, \langle s_1^\tau, \dots, s_{k_\tau}^\tau \rangle, \dots \rangle. \quad (11)$$

This hierarchical decomposition is used solely as an analysis tool to describe the structure of reasoning trajectories.

B.5 Accuracy-Cost Efficiency

We introduce *Accuracy-Cost Efficiency (ACE)*, a metric that measures how efficiently a model’s generation leads to a correct final answer under hybrid SYSTEM 1/2 reasoning. ACE extends prior efficiency metrics, including *Outcome Efficiency* [15], *VT* [22], and *Efficiency* [59], to hybrid reasoning settings. Unlike these measures, ACE accommodates optional or partially expressed reasoning and does not assume that correct answers are embedded within a single continuous reasoning trace.

Let N denote the number of test instances. For each instance i , let $\sigma_i \in \{0, 1\}$ indicate whether the final answer is correct, τ_i denote the total number of generated tokens, and $\hat{\tau}_i$ denote the position of the first token where the correct answer appears. Let $m_i^{\text{Think}} \in \{0, 1\}$ indicate whether the model produces explicit reasoning.

In hybrid reasoning scenarios, the correct answer may not appear explicitly in the reasoning trace or may be produced without any explicit reasoning. In such cases, we set $\hat{\tau}_i = \tau_i$, corresponding to maximal efficiency under ACE, since no redundant reasoning tokens are generated beyond the final answer.

Per-Instance Efficiency. Based on the above definitions, the per-instance efficiency is computed:

$$\eta_i = 1 - m_i^{\text{Think}} \mathbb{I}[\hat{\tau}_i < \tau_i] \left(1 - \frac{\hat{\tau}_i}{\tau_i}\right), \quad (12)$$

where:

- $\eta_i = 1$ if either no explicit reasoning is generated ($m_i^{\text{Think}} = 0$) or the correct answer first appears at the end of the trajectory ($\hat{\tau}_i = \tau_i$),
- $\eta_i < 1$ if explicit reasoning continues beyond the first correct token, decreasing proportionally to the fraction of redundant reasoning tokens.

Overall ACE Score. Finally, the overall ACE score is computed:

$$\xi_{\text{ACE}} = \frac{1}{N} \sum_{i=1}^N \sigma_i \eta_i. \quad (13)$$

This score is bounded, $\xi_{\text{ACE}} \in [0, 1]$, and is used solely as an evaluation metric. Incorrect predictions contribute zero to the score, while correct predictions achieve maximal efficiency if no redundant reasoning occurs. If explicit reasoning continues beyond the earliest correct answer, the score decreases proportionally to the fraction of redundant reasoning tokens, $1 - \hat{\tau}_i/\tau_i$.

C Additional Experimental Results

C.1 Analysis of Token Distribution

Figure 5 shows the distributions of token usage by reasoning mode and response correctness. For correct instances (Left), NOTHINK peaks sharply at low token counts, reflecting SYSTEM 1 efficiency, whereas THINK consumes more tokens even when correct, revealing overthinking. For incorrect instances (Right), THINK exhibits a heavy-tailed distribution reaching the context limit, demonstrating the high cost of failed SYSTEM 2 reasoning.

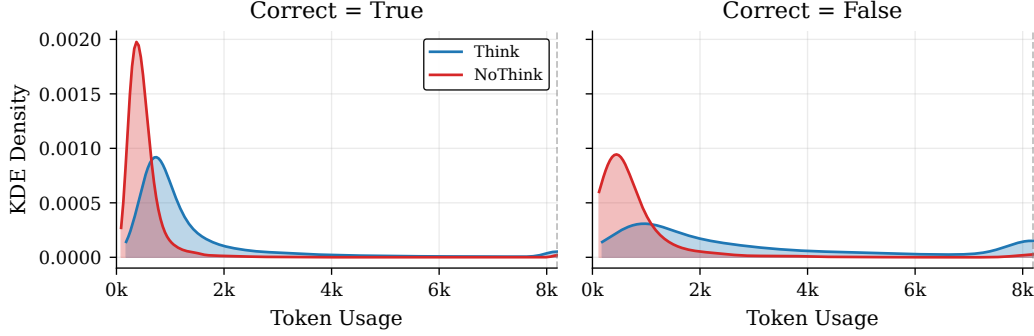


Figure 5: **When2Think token usage distributions by reasoning mode.** Kernel Density Estimation (KDE) of generated token counts for THINK (blue) and NOTHINK (red), stratified by response correctness. **Left:** Correct responses—NOTHINK peaks at low token usage. **Right:** Incorrect responses—THINK exhibits a broader distribution with a long tail, indicating higher token consumption during failed reasoning.

C.2 Reasoning Mode Performance

Table 4: **Detailed Performance by Reasoning Mode on GSM-Plus.** The table reports Pass@3 accuracy (%), average token usage ($\downarrow$ Tokens), and step counts ($\downarrow$ Steps) for the THINK mode, alongside accuracy, token usage, and selection ratio for the NOTHINK mode. All metrics are reported as mean $\pm$ standard deviation over five runs.

Benchmark	GSM-Plus					
	THINK			NOTHINK		
	$\uparrow$ Pass@3(%)	$\downarrow$ Tokens(#)	$\downarrow$ Steps(#)	$\uparrow$ Pass@3(%)	$\downarrow$ Tokens(#)	Ratio(%)
1.5B Models						
AdaptThink-delta0.05	67.4 $\pm$ 1.6	1,807 $\pm$ 59	10 $\pm$ 1	85.9 $\pm$ 0.2	561 $\pm$ 16	87.5 $\pm$ 0.1
Thinkless-RL	84.5 $\pm$ 0.2	1,872 $\pm$ 30	14 $\pm$ 0	93.7 $\pm$ 0.8	1,378 $\pm$ 62	14.7 $\pm$ 0.5
When2Think (Ours)	80.8 $\pm$ 0.3	1,401 $\pm$ 27	21 $\pm$ 1	93.0 $\pm$ 0.5	538 $\pm$ 15	40.3 $\pm$ 0.3

Table 5: **Detailed Performance by Reasoning Mode on Minerva.** The table reports Pass@3 accuracy (%), average token usage ($\downarrow$ Tokens), and step counts ($\downarrow$ Steps) for the THINK mode, alongside accuracy, token usage, and selection ratio for the NOTHINK mode. All metrics are reported as mean $\pm$ standard deviation over five runs.

Benchmark	Minerva					
	THINK			NOTHINK		
	$\uparrow$ Pass@3(%)	$\downarrow$ Tokens(#)	$\downarrow$ Steps(#)	$\uparrow$ Pass@3(%)	$\downarrow$ Tokens(#)	Ratio(%)
1.5B Models						
AdaptThink-delta0.05	36.6 $\pm$ 1.6	3,012 $\pm$ 285	34 $\pm$ 6	57.3 $\pm$ 1.6	1,154 $\pm$ 96	58.7 $\pm$ 2.6
Thinkless-RL	52.5 $\pm$ 2.0	4,982 $\pm$ 200	74 $\pm$ 7	47.4 $\pm$ 10.1	6,487 $\pm$ 1,603	5.5 $\pm$ 1.1
When2Think (Ours)	50.3 $\pm$ 2.5	3,282 $\pm$ 82	92 $\pm$ 3	78.8 $\pm$ 4.3	811 $\pm$ 192	9.0 $\pm$ 0.6

Table 6: **Detailed Performance by Reasoning Mode on OlympiadBench.** The table reports Pass@3 accuracy (%), average token usage ($\downarrow$ Tokens), and step counts ($\downarrow$ Steps) for the THINK mode, alongside accuracy, token usage, and selection ratio for the NOTHINK mode. All metrics are reported as mean $\pm$ standard deviation over five runs.

Benchmark \ Model		OlympiadBench-Math					
		THINK			NOTHINK		
		↑ Pass@3(%)	↓ Tokens(#)	↓ Steps(#)	↑ Pass@3(%)	↓ Tokens(#)	Ratio(%)
1.5B Models							
AdaptThink-delta0.05		41.5 ± 2.1	7,223 ± 215	69 ± 4	62.5 ± 1.3	2,003 ± 54	80.5 ± 0.9
Thinkless-RL		63.6 ± 0.9	6,839 ± 116	117 ± 4	59.4 ± 9.0	9,785 ± 979	4.0 ± 0.7
When2Think (Ours)		57.5 ± 1.2	6,635 ± 57	204 ± 3	90.5 ± 2.1	1,086 ± 93	14.7 ± 0.5

Table 7: **Detailed Performance by Reasoning Mode on AIME24.** The table reports Pass@3 accuracy (%), average token usage ($\downarrow$ Tokens), and step counts ($\downarrow$ Steps) for the THINK mode, alongside accuracy, token usage, and selection ratio for the NOTHINK mode. All metrics are reported as mean $\pm$ standard deviation over five runs.

Benchmark \ Model		AIME24					
		THINK			NOTHINK		
		↑ Pass@3(%)	↓ Tokens(#)	↓ Steps(#)	↑ Pass@3(%)	↓ Tokens(#)	Ratio(%)
<i>1.5B Models</i>							
AdaptThink-delta0.05		44.0 ± 11.9	8,121 ± 678	338 ± 72	47.6 ± 12.8	3,227 ± 552	48.0 ± 11.7
Thinkless-RL		47.7 ± 4.6	10,861 ± 245	221 ± 31	0.0 ± 0.0	18,897 ± 1,606	2.0 ± 1.8
When2Think (Ours)		56.0 ± 2.8	10,236 ± 438	399 ± 27	—	—	0.0 ± 0.0

Table 8: **Detailed Performance by Reasoning Mode on AIME25.** The table reports Pass@3 accuracy (%), average token usage ($\downarrow$ Tokens), and step counts ($\downarrow$ Steps) for the THINK mode, alongside accuracy, token usage, and selection ratio for the NOTHINK mode. All metrics are reported as mean $\pm$ standard deviation over five runs.

Benchmark \ Model		AIME25					
		THINK			NOTHINK		
		↑ Pass@3(%)	↓ Tokens(#)	↓ Steps(#)	↑ Pass@3(%)	↓ Tokens(#)	Ratio(%)
1.5B Models							
AdaptThink-delta0.05		29.7 ± 12.6	9,114 ± 828	345 ± 30	32.7 ± 3.3	3,000 ± 585	36.7 ± 4.1
Thinkless-RL		35.2 ± 8.9	10,667 ± 729	210 ± 35	0.0 ± 0.0	19,497 ± 2,328	4.7 ± 5.1
When2Think (Ours)		40.0 ± 0.0	9,549 ± 719	424 ± 65	—	—	0.0 ± 0.0

C.3 Performance Across Benchmarks

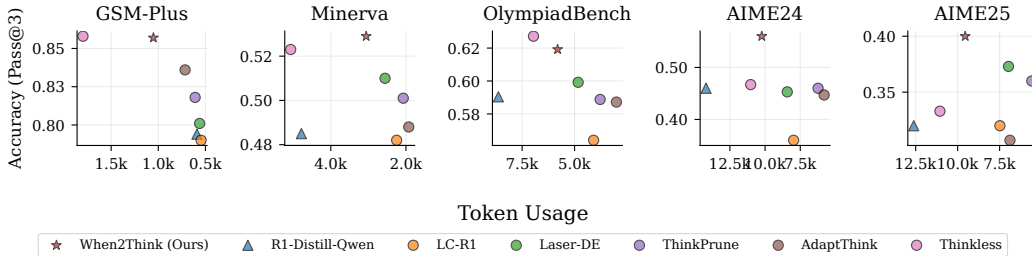


Figure 6: **Accuracy-Efficiency trade-off across reasoning benchmarks.** Performance of **When2Think (Ours)** ($\star$) is compared with the base model **R1-Distill-Qwen** (Triangle) on five benchmarks of varying difficulty: GSM-Plus, Minerva, OlympiadBench, AIME24, and AIME25. The x-axis shows **Token Usage** (reversed; further right indicates fewer tokens/more efficient), and the y-axis shows **Pass@3 Accuracy**. Across benchmarks, **When2Think** points generally occupy the top-right region relative to the baseline, indicating higher accuracy with lower token usage.

Table 9: **Extended performance comparison on reasoning benchmarks.** The table reports Pass@3 accuracy (%), average tokens per response ($\downarrow$ Tokens), and Accuracy-Cost Efficiency ($\uparrow$ ACE) on GSM-Plus, Minerva, OlympiadBench-Math, AIME24, and AIME25 under a unified evaluation protocol. Results are obtained using a sampling-based Pass@k procedure with matched decoding settings and are reported as mean $\pm$ standard deviation over five runs. Values in parentheses indicate differences relative to R1-Distill-Qwen (**green**: improvement; **red**: degradation).

Model	Benchmark	GSM-Plus			Minerva			OlympiadBench-Math			AIME24			AIME25		
		↑ Pass@3(%)	↓ Tokens(10 ⁶)	↑ ACE(%)	↑ Pass@3(%)	↓ Tokens(10 ⁶)	↑ ACE(%)	↑ Pass@3(%)	↓ Tokens(10 ⁶)	↑ ACE(%)	↑ Pass@3(%)	↓ Tokens(10 ⁶)	↑ ACE(%)	↑ Pass@3(%)	↓ Tokens(10 ⁶)	↑ ACE(%)
1.5B Models																
R1-Distill-Qwen (Base)	Qwen2.5-Math-Instruct	73.6 ± 0.3	346 ± 2.9	—	30.1 ± 1.5	71.6 ± 53.2	—	29.6 ± 0.9	976 ± 31.0	—	8.0 ± 3.8	1,373 ± 313.1	—	1.3 ± 3.0	995 ± 103.4	—
	Qwen2.5-Math-Instruct	84.3 ± 0.3	360 ± 1.8	—	44.8 ± 0.5	67.1 ± 14.7	—	53.2 ± 0.8	842 ± 8.3	—	22.7 ± 1.5	1,057 ± 57.7	—	19.3 ± 2.8	925 ± 64.8	—
	DeepScaleR-Preview	85.4 ± 0.3	1,358 ± 13.4	83.7 ± 0.1	55.4 ± 1.1	4,130 ± 56.4	43.7 ± 1.0	62.8 ± 0.8	5,215 ± 43.3	42.9 ± 0.5	58.0 ± 5.6	8,473 ± 479.5	38.4 ± 4.4	39.3 ± 6.8	8,074 ± 288.6	17.7 ± 3.1
	LC-R1	79.0 ± 0.3	546 ± 9.9	79.0 ± 0.3	48.2 ± 1.7	2,248 ± 71.8	42.7 ± 1.8	55.5 ± 1.1	4,092 ± 113.0	39.9 ± 1.1	36.0 ± 4.9	7,972 ± 1,024.3	23.0 ± 3.4	32.0 ± 3.8	6,971 ± 278.7	18.5 ± 2.7
	ThinkPrime-ier7k	81.8 ± 0.3	609 ± 5.5	79.3 ± 0.3	50.1 ± 1.9	2,072 ± 49.2	32.9 ± 1.5	58.6 ± 0.7	3,781 ± 66.5	35.7 ± 0.6	46.0 ± 4.3	6,256 ± 555.4	27.0 ± 2.0	36.0 ± 3.7	5,582 ± 273.3	21.3 ± 1.0
Laser-DE-L4096	AdaptThink-delta0.05	80.1 ± 0.2	592 ± 4.1	79.4 ± 0.3	51.0 ± 1.2	2,556 ± 105.5	38.7 ± 0.7	59.9 ± 1.1	4,830 ± 43.4	39.1 ± 1.0	45.3 ± 3.8	8,425 ± 362.3	27.0 ± 2.6	37.3 ± 8.0	6,971 ± 1,024.3	24.1 ± 6.6
	Thinkless-RL	83.6 ± 0.2	716 ± 17.5	83.5 ± 0.2	48.8 ± 1.3	1,925 ± 110.9	48.0 ± 1.5	58.4 ± 0.8	3,004 ± 76.2	56.9 ± 0.9	44.7 ± 4.5	5,806 ± 438.8	40.1 ± 4.9	30.7 ± 8.0	6,883 ± 440.2	25.1 ± 5.7
	AdaptThink-delta0.05	85.8 ± 0.2	1,799 ± 16.2	82.7 ± 0.2	52.3 ± 1.4	5,067 ± 166.0	44.3 ± 1.4	63.4 ± 1.1	6,990 ± 76.2	47.5 ± 1.5	46.7 ± 4.1	11,023 ± 200.4	30.4 ± 4.4	33.3 ± 7.5	11,056 ± 61.7	21.4 ± 4.7
	Thinkless-RL	85.7 ± 0.3	1,209 ± 17.9	82.7 ± 0.2	52.3 ± 1.4	2,857 ± 105.5	44.3 ± 1.4	63.4 ± 1.1	6,990 ± 76.2	47.5 ± 1.5	46.7 ± 4.1	11,023 ± 200.4	30.4 ± 4.4	33.3 ± 7.5	11,056 ± 61.7	21.4 ± 4.7
	When2Think (Ours)	85.7 ± 0.3	1,052 ± 17.9	80.7 ± 0.4	52.9 ± 2.1	3,058 ± 58.6	40.6 ± 1.7	62.4 ± 0.9	5,009 ± 56.4	43.9 ± 1.2	56.0 ± 2.8	10,236 ± 438.0	32.1 ± 1.8	40.0 ± 6.0	9,549 ± 719.4	24.6 ± 1.1
7B Models																
R1-Distill-Qwen	Qwen2.5-Math-Instruct	88.8 ± 0.2	342 ± 0.2	—	54.0 ± 1.4	634 ± 1.4	—	52.3 ± 1.4	819 ± 11.0	—	18.0 ± 3.8	1,087 ± 107.0	—	20.7 ± 7.2	937 ± 13.5	—
	Qwen2.5-Math-Instruct	89.1 ± 0.2	421 ± 3.1	—	51.6 ± 1.2	833 ± 11.3	—	55.8 ± 0.5	1,110 ± 13.4	—	24.0 ± 3.8	1,614 ± 140.6	—	22.0 ± 4.5	1,512 ± 71.5	—
	DeepScaleR-Preview	87.6 ± 0.1	521 ± 1.1	87.5 ± 0.1	63.6 ± 1.1	3,517 ± 299.5	50.0 ± 1.7	65.6 ± 0.5	4,984 ± 66.6	49.3 ± 1.1	60.0 ± 8.5	7,266 ± 554.0	39.3 ± 6.3	43.3 ± 4.7	6,611 ± 473.3	29.1 ± 4.6
	Laser-DE-L4096	90.4 ± 0.1	849 ± 1.3	87.0 ± 0.2	61.9 ± 1.0	1,767 ± 29.4	51.8 ± 1.0	71.1 ± 0.6	3,358 ± 21.7	46.8 ± 0.6	66.7 ± 2.4	5,563 ± 283.0	42.2 ± 2.2	52.0 ± 5.6	5,533 ± 176.2	35.0 ± 4.1
	AdaptThink-delta0.05	88.8 ± 0.2	394 ± 2.0	88.8 ± 0.2	62.0 ± 1.6	1,983 ± 67.2	56.6 ± 2.1	67.2 ± 1.0	4,080 ± 79.7	56.4 ± 0.6	68.0 ± 3.8	6,682 ± 762.7	48.8 ± 2.2	46.7 ± 2.4	6,599 ± 498.6	33.8 ± 2.3
When2Think (Ours)	AdaptThink-delta0.05	89.3 ± 0.1	460 ± 2.3	88.7 ± 0.1	63.2 ± 1.1	2,451 ± 47.5	54.1 ± 0.8	69.1 ± 0.7	4,250 ± 70.6	55.4 ± 0.8	68.7 ± 4.5	7,855 ± 397.9	45.4 ± 3.7	48.0 ± 1.8	7,347 ± 790.1	31.6 ± 1.4
	Thinkless-RL	89.3 ± 0.1	460 ± 2.3	88.7 ± 0.1	63.2 ± 1.1	2,451 ± 47.5	54.1 ± 0.8	69.1 ± 0.7	4,250 ± 70.6	55.4 ± 0.8	68.7 ± 4.5	7,855 ± 397.9	45.4 ± 3.7	48.0 ± 1.8	7,347 ± 790.1	31.6 ± 1.4
	When2Think (Ours)	89.3 ± 0.1	460 ± 2.3	88.7 ± 0.1	63.2 ± 1.1	2,451 ± 47.5	54.1 ± 0.8	69.1 ± 0.7	4,250 ± 70.6	55.4 ± 0.8	68.7 ± 4.5	7,855 ± 397.9	45.4 ± 3.7	48.0 ± 1.8	7,347 ± 790.1	31.6 ± 1.4
	Thinkless-RL	89.3 ± 0.1	460 ± 2.3	88.7 ± 0.1	63.2 ± 1.1	2,451 ± 47.5	54.1 ± 0.8	69.1 ± 0.7	4,250 ± 70.6	55.4 ± 0.8	68.7 ± 4.5	7,855 ± 397.9	45.4 ± 3.7	48.0 ± 1.8	7,347 ± 790.1	31.6 ± 1.4
	When2Think (Ours)	89.3 ± 0.1	460 ± 2.3	88.7 ± 0.1	63.2 ± 1.1	2,451 ± 47.5	54.1 ± 0.8	69.1 ± 0.7	4,250 ± 70.6	55.4 ± 0.8	68.7 ± 4.5	7,855 ± 397.9	45.4 ± 3.7	48.0 ± 1.8	7,347 ± 790.1	31.6 ± 1.4

Table 10: **Performance comparison on MATH-500 stratified by problem difficulty (Levels 1–5).** The table reports Pass@1 accuracy (%), average tokens per response ($\downarrow$ Tokens), and Accuracy–Cost Efficiency ($\uparrow$ ACE) for each difficulty level under the same evaluation protocol as the main results. Results are reported as mean $\pm$ standard deviation over five runs. Values in parentheses indicate differences relative to R1-Distill-Qwen (green: improvement; red: degradation).

Model	Benchmark	MATH-500				
		Level 1	Level 2	Level 3	Level 4	Level 5
		$\uparrow$ Pass@1 (%) $\downarrow$ Tokens#	$\uparrow$ Pass@1 (%) $\downarrow$ Tokens#	$\uparrow$ Pass@1 (%) $\downarrow$ Tokens#	$\uparrow$ Pass@1 (%) $\downarrow$ Tokens#	$\uparrow$ Pass@1 (%) $\downarrow$ Tokens#
1.5B Models						
Qwen2.5-Instruct		79.5 $\pm$ 6.9	342 $\pm$ 11.9	–	71.1 $\pm$ 4.2	429 $\pm$ 15.8
Qwen2.5-Math-Instruct		94.0 $\pm$ 2.1	346 $\pm$ 6.1	–	90.2 $\pm$ 0.5	422 $\pm$ 4.4
R1-Distill-Qwen (Base)		92.1 $\pm$ 2.7	1,199 $\pm$ 100.0	89.3 $\pm$ 2.1	91.8 $\pm$ 2.0	1,724 $\pm$ 106.2
DeepScaler-Preview		94.0 $\pm$ 2.1	1,250 $\pm$ 02.6	86.3 $\pm$ 2.5	95.8 $\pm$ 0.5	1,533 $\pm$ 48.4
L-C-R1		90.2 $\pm$ 3.8	626 $\pm$ 23.2	85.6 $\pm$ 3.3	85.8 $\pm$ 4.0	916 $\pm$ 74.5
ThinkPrune-Iter3k		93.0 $\pm$ 5.5	(-5.78) 770 $\pm$ 20.3	(-3.77) 80.9 $\pm$ 5.5	(-6.09) 91.3 $\pm$ 2.9	(-8.08) 990 $\pm$ 37.3
Laser-DE-L4096		91.2 $\pm$ 1.9	(+0.39) 898 $\pm$ 68.1	(-5.39) 82.7 $\pm$ 2.7	(-6.39) 90.2 $\pm$ 1.8	(-7.34) 1,211 $\pm$ 23.1
AdaptThink-delta0.05		94.4 $\pm$ 2.1	(+2.35) 512 $\pm$ 186.0	(-6.67) 94.4 $\pm$ 2.1	(-0.14) 92.0 $\pm$ 1.4	(-1.214) 510 $\pm$ 20.7
Thinkless-RL		95.8 $\pm$ 3.4	(+3.77) 1,374 $\pm$ 106.8	(+1.75) 90.8 $\pm$ 4.1	(+1.43) 94.7 $\pm$ 2.7	(+1.13) 1,867 $\pm$ 166.3
When2Think (Ours)		95.8 $\pm$ 1.9	(+3.37) 619 $\pm$ 11.0	(-5.80) 94.3 $\pm$ 1.9	(+1.1) 92.9 $\pm$ 2.6	(-8.62) 882 $\pm$ 76.1
7B Models						
Qwen2.5-Instruct		96.3 $\pm$ 1.3	342 $\pm$ 4.0	–	92.4 $\pm$ 1.4	440 $\pm$ 4.2
Qwen2.5-Math-Instruct		96.7 $\pm$ 2.7	370 $\pm$ 16.0	–	93.8 $\pm$ 1.0	449 $\pm$ 7.7
R1-Distill-Qwen		93.5 $\pm$ 1.0	948 $\pm$ 83.4	90.1 $\pm$ 2.2	93.3 $\pm$ 1.1	1,306 $\pm$ 33.5
Laser-DE-L4096		96.7 $\pm$ 1.3	(+3.2) 953 $\pm$ 10.0	(-7.25) 82.9 $\pm$ 1.5	(+4.0) 97.3 $\pm$ 1.3	(-2.26) 1,080 $\pm$ 21.8
AdaptThink-delta0.05		95.3 $\pm$ 1.6	(+1.8) 420 $\pm$ 12.7	(-5.28) 95.2 $\pm$ 1.7	(+2.33) 95.6 $\pm$ 1.8	(-7.38) 568 $\pm$ 16.8
When2Think (Ours)		96.7 $\pm$ 2.1	(+3.2) 603 $\pm$ 23.0	(-3.16) 95.7 $\pm$ 2.3	(+5.34) 98.7 $\pm$ 0.9	(-4.02) 904 $\pm$ 33.8
						93.1 $\pm$ 1.2
						97.0 $\pm$ 1.4
						1,247 $\pm$ 67.0
						91.4 $\pm$ 1.8
						93.4 $\pm$ 1.2
						2,082 $\pm$ 36.8
						83.2 $\pm$ 1.0
						87.6 $\pm$ 2.0
						3,825 $\pm$ 101.6
						76.9 $\pm$ 1.4

D Ablation Studies

D.1 Reward and Advantage Landscapes under IDAC and BWS

BWS versus GRPO in the Advantage Landscape. We analyze why Batch-Wise Standardization (BWS) is critical for optimizing IDAC by comparing three reward formulations: the raw THINK reward, BWS advantages, and GRPO-style group-relative advantages. We construct a Monte Carlo simulation using offline statistics from R1-Distill-Qwen-1.5B on DeepScaleR. For each problem, empirical accuracy α captures difficulty and reference length τ approximates typical reasoning cost. We then generate THINK trajectories by perturbing τ with log-normal noise and sampling correctness according to α . This setup isolates how different normalization schemes allocate credit to reasoning trajectories as a function of problem difficulty and token usage.

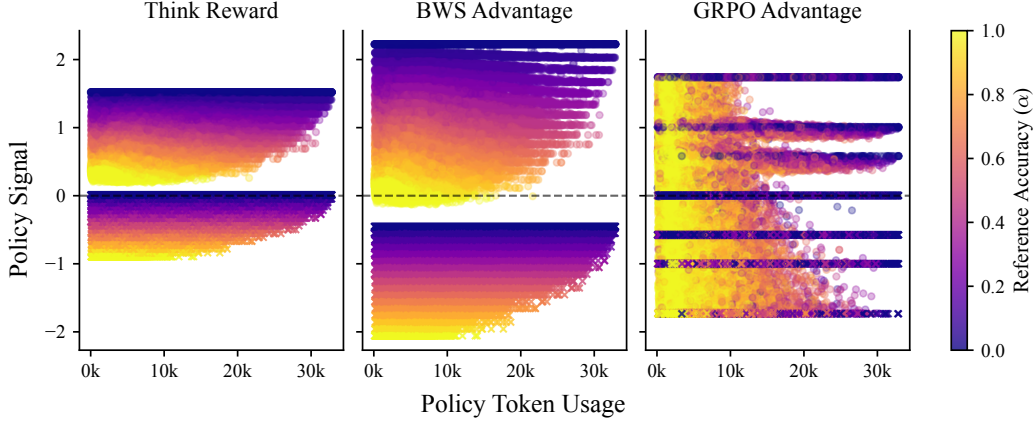


Figure 7: **Empirically Grounded Advantage Landscape of Explicit Reasoning.** Policy signals for simulated THINK trajectories constructed from R1-Distill-Qwen-1.5B statistics on DeepScaleR. Colors represent reference accuracy α (hard problems in blue, $\alpha \rightarrow 0$, to easy problems in yellow, $\alpha \rightarrow 1$), and markers indicate correctness (circles for correct, crosses for incorrect). **Left:** Raw IDAC THINK reward with absolute length-sensitive bonus. **Middle:** BWS advantage (ours), which preserves positive advantages for long successful trajectories on hard problems. **Right:** GRPO-style advantage, where problem instance normalization yields fragmented signals and horizontal banding.

BWS: Preserving Global Difficulty Signals. BWS converts raw IDAC rewards into relative advantages computed over the full mini-batch, rather than normalizing within each instance. This batch-wise formulation preserves cross-instance difficulty structure that is lost under instance-local normalization. As a result, trajectories that solve difficult problems can remain above the batch expectation and receive positive advantages, even when their absolute rewards are reduced. As shown in Figure 7, this effect is most evident for low- α instances, where successful long trajectories retain positive optimization signals despite high token usage. Thus, BWS does not merely reduce variance; it preserves global difficulty signals in the advantage function, enabling credit assignment for deliberate but necessary long-horizon reasoning.

GRPO: Removing Cross-Instance Reward Structure. In contrast, GRPO-style normalization computes advantages relative only to rollouts of the same problem, removing any cross-instance reward comparisons. While this eliminates instance-level scale variation, it also discards the global difficulty structure encoded in IDAC. As a consequence, the advantage landscape becomes fragmented, manifesting as horizontal bands in Figure 7. Because normalization is restricted to within-problem rollouts, credit assignment becomes sensitive to the composition of sampled trajectories. In particular, long but successful reasoning traces on difficult problems may receive weak or unstable signals when competing against short incorrect or prematurely terminated rollouts. These results highlight a key limitation of instance-local normalization, and motivate BWS as a mechanism for preserving cross-instance difficulty structure in critic-free optimization.

D.2 Analysis of BWS Training Dynamics

We analyze training dynamics to evaluate the effectiveness of our optimization strategy, focusing on the role of *Batch-Wise Standardization* (BWS). Figure 8 reports both task performance and reasoning behavior throughout post-training.

Improved Convergence and Stability. BWS substantially stabilizes policy optimization. Without BWS, raw reward signals exhibit high variance across instances, leading to noisy gradient estimates and unstable learning trajectories. By normalizing rewards using batch-level statistics (μ_k, σ_k), BWS reduces gradient variance and disentangles instance difficulty from policy quality in the optimization signal. This effect is particularly pronounced on AIME24, where problem difficulty varies sharply; under BWS, the model consistently improves performance even on the hardest instances.

Controlled Reasoning Depth. BWS also plays a critical role in regulating reasoning length. Without proper normalization, baseline policies tend to drift toward degenerate behaviors—either collapsing to minimal reasoning or producing excessively verbose outputs—due to unstable credit assignment. In contrast, *When2Think* maintains a stable *Think Ratio* and *Response Length*, gradually adapting reasoning depth to optimize the accuracy–efficiency trade-off. Notably, this behavior emerges without relying on an explicit value function, indicating that BWS effectively stabilizes critic-free policy optimization.

Sample Efficiency and Difficulty Scaling. Our approach exhibits improved sample efficiency. On AIME24, *When2Think* reaches peak performance in approximately half the training steps required by the baseline. Moreover, the performance gap widens as task difficulty increases, highlighting robustness under high-entropy reasoning regimes. Finally, BWS decouples response length from accuracy, preventing the *reasoning bloat* observed in the baseline, where additional tokens fail to yield meaningful performance gains.

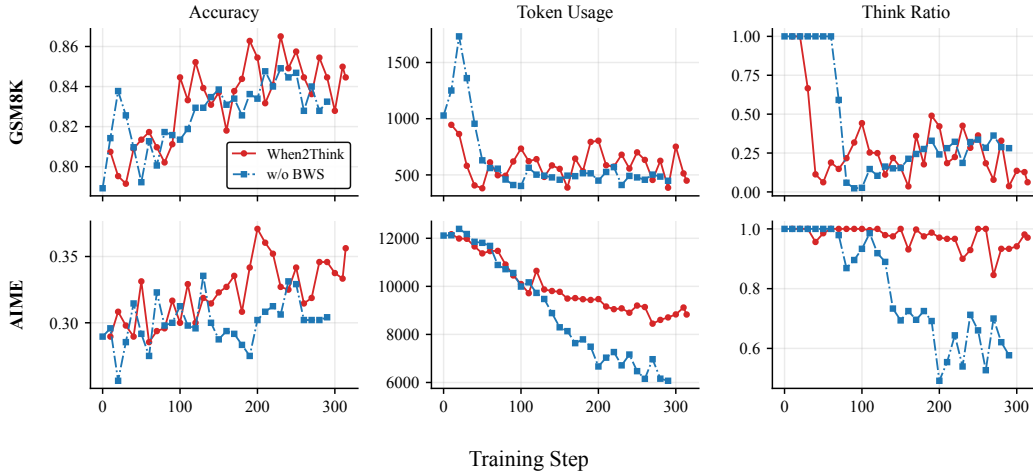


Figure 8: **Training Dynamics under Batch-Wise Standardization Ablations.** *When2Think* (red, with batch-wise standardized advantage) and *Baseline* (blue, without standardization) on DeepScaleR. **Top:** GSM8K (easy). **Bottom:** AIME24 (hard). Each row reports validation accuracy, average token usage, and Think Ratio over training steps.

D.3 Analysis of IS & IDAC Training Dynamics

We further analyze the roles of importance sampling (IS) and Instance-level Difficulty-Aware Control (IDAC) during training. We compare three variants: the full *When2Think* model, a variant without IS (retaining IDAC and BWS), and a variant without IDAC (retaining IS and BWS). This ablation isolates whether the observed accuracy–efficiency trade-offs are driven by balanced THINK/NOTHINK exploration (via IS) or by difficulty-aware reward shaping and normalized optimization signals (via IDAC and BWS).

IDAC and BWS provide the main depth-control signal. The *IDAC+BWS* variant removes the balanced exploration distribution induced by IS, while retaining difficulty-aware reward shaping and batch-wise standardized advantages. As shown in Figure 9, this variant remains competitive in validation accuracy, especially on the harder AIME24 benchmark. This suggests that the main depth-control signal is primarily driven by IDAC-based reward shaping together with BWS-based advantage standardization, rather than by importance sampling alone.

IS shifts the policy toward a more efficient hybrid operating point. Although IS is not the primary source of the depth-control signal, it changes the operating point of the learned policy. On the easier GSM8K benchmark, the full *When2Think* model achieves comparable validation accuracy while reducing both token usage and Think Ratio relative to the *IDAC+BWS* variant. This indicates that IS helps expose and exploit the NOTHINK mode during training, encouraging more efficient behavior when explicit reasoning is unnecessary.

IDAC is necessary for difficulty-aware depth control. The *IS+BWS* variant retains balanced mode exploration but removes IDAC. While this variant can still learn to switch between THINK and NOTHINK, it lacks the difficulty-aware signal that determines how much reasoning should be allocated once THINK is selected. As a result, IS alone can encourage mode switching, but does not provide the same control over reasoning depth across easy and hard instances.

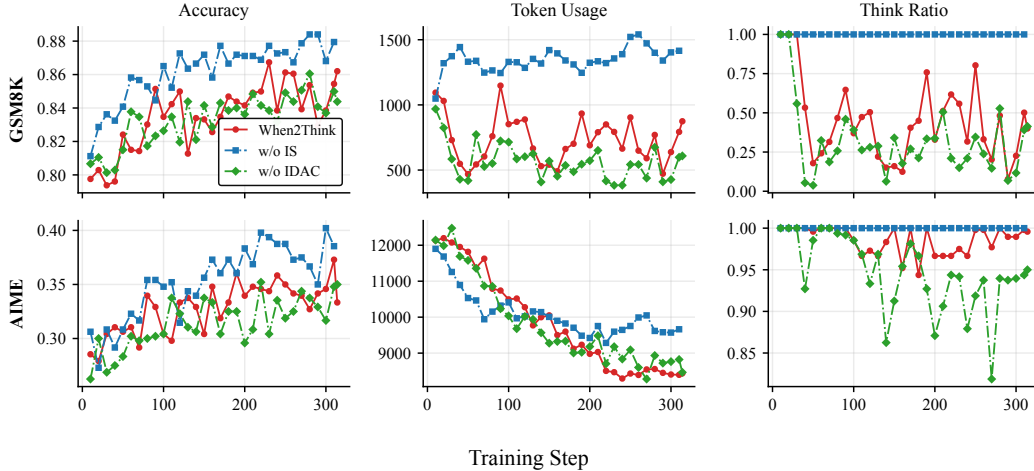


Figure 9: **Training Dynamics under IS, IDAC Ablations.** Training dynamics of three variants on DeepScaleR: the full *When2Think* model (IS + IDAC + BWS), a variant without importance sampling (IDAC + BWS), and a variant without IDAC (IS + BWS). **Top:** GSM8K (easy). **Bottom:** AIME24 (hard). Each row reports validation accuracy, token usage, and Think Ratio over training steps.

D.4 Ablation on Hybrid Bonus Design

Separate bonuses provide limited benefit. As shown in Figure 10, varying the NO THINK bonus η does not yield a clear improvement in the accuracy–efficiency trade-off. In particular, configurations with larger η tend to produce lower validation accuracy, suggesting that overly strong incentives for NO THINK can bias the policy toward direct answering even when explicit reasoning remains useful. This behavior is consistent with hard-instance under-allocation: encouraging NO THINK too aggressively can suppress necessary long-horizon reasoning.

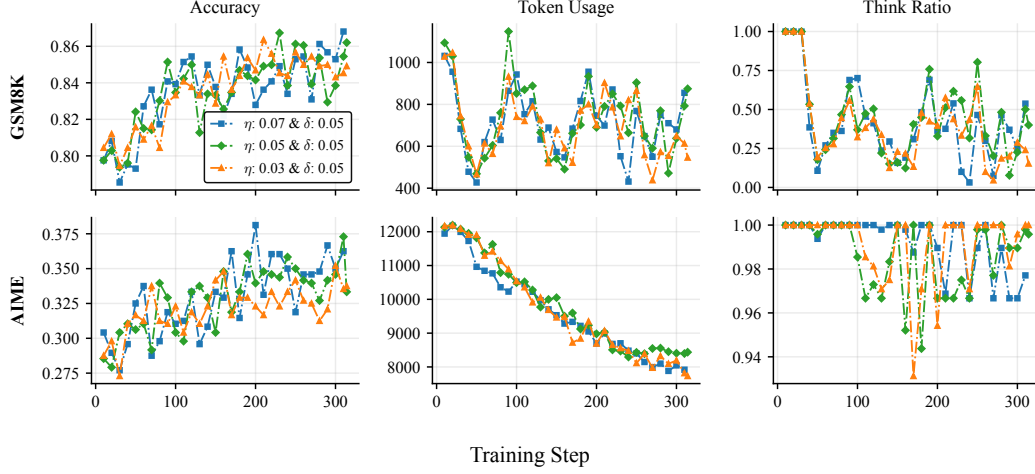


Figure 10: **Training Dynamics under η and δ Variations.** Training dynamics of *When2Think* under different η settings on DeepScaleR, with δ fixed to 0.05. **Top:** GSM8K (easy). **Bottom:** AIME24 (hard). Each row reports validation accuracy, token usage, and Think Ratio over training steps.

Unified bonus yields more stable training. We therefore simplify the reward by using a single correctness-gated efficiency coefficient δ for both modes, with DADS controlling how strongly the bonus is applied to THINK trajectories. The training dynamics in Figure 11 show that smaller unified bonus values lead to more stable behavior, avoiding both excessive NO THINK collapse and overly verbose THINK trajectories. This suggests that the reward does not require separate mode-specific bonus coefficients.

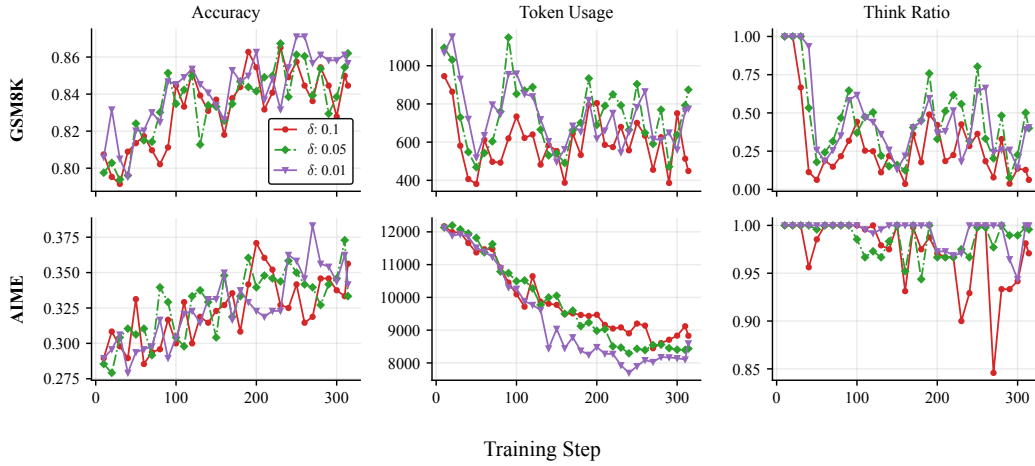


Figure 11: **Training Dynamics under δ Variations.** Training dynamics of *When2Think* under different efficiency-bonus coefficients δ on DeepScaleR. **Top:** GSM8K (easy). **Bottom:** AIME24 (hard). Each row reports validation accuracy, token usage, and Think Ratio over training steps.

D.5 Effectiveness of the Dual Math Verifier

We analyze the contribution of the Dual Math Verifier through subject-level performance on MATH-500. Figure 12 compares the Base Model (R1-Distill), the Baseline with standard verification, and our proposed method (Improved). Figure 12(c) further confirms that each subject exhibits sufficient coverage in both problem count and difficulty distribution, enabling meaningful comparison.

Robustness to Non-Numeric Answers. Subjects such as *Geometry* and *Precalculus* contain a high proportion of symbolic, structured, or text-heavy answers. In these categories, the Baseline exhibits notable performance degradation due to the inherent limitations of string-based verification, which fails to recognize semantically equivalent but syntactically distinct solutions (e.g., reordered vector coordinates or algebraically equivalent expressions). By incorporating symbolic equivalence checking, the Dual Math Verifier effectively resolves this mismatch, substantially improving robustness to non-numeric answer formats and yielding consistent gains across these subjects.

Trade-offs in Intermediate Algebra. Performance gains in *Intermediate Algebra* are comparatively smaller. This behavior is not attributable to data scarcity, as models such as DeepScaleR—trained on the same dataset—achieve stronger results, but rather reflects a trade-off between verification strictness and recall. In particular, complex multi-step derivations involving alternative but equivalent forms (e.g., factored versus expanded polynomials) may be falsely rejected under strict symbolic parsing. Moreover, efficiency constraints imposed by IDAC can limit the allocation of reasoning steps required for such transformations. These observations suggest that subject-aware verification tolerance or adaptive length modulation may further improve performance in algebra-intensive domains.

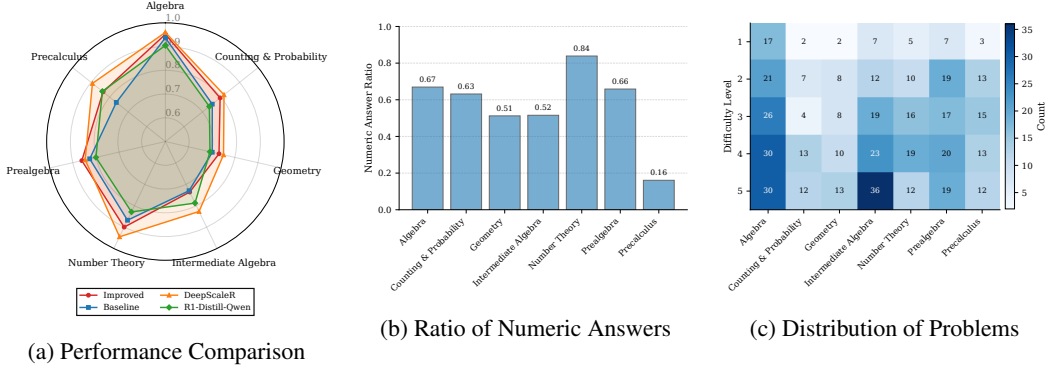


Figure 12: **Subject-wise analysis of MATH-500.** (a) Performance across subjects. The proposed method (Improved) recovers baseline drops in geometry and precalculus. (b) Fraction of numeric-answer problems; low numeric ratios align with baseline failures. (c) Heatmap of problem counts across difficulty levels (1–5), confirming sufficient sample coverage for each subject.

E Case Studies

E.1 Overthinking on Easy Problem

[GSM-Plus; R1-Distill-Qwen-1.5B] Correct Overthinking Response (32,206 tokens)

Comet Halley, which is approximately 15 kilometers wide, orbits the sun every 75 years. Bill's dad saw the Comet when he was 30 years old. Bill, who has a brother 5 years younger than him, saw the comet a second time when he was three times the age his father was when he saw the Comet. Despite the fact that Bill's favorite number is 7, how old was Bill when he saw the Comet for the first time?

<THINK>

But according to the problem, the second appearance is at 90, which is 60 years after the first, which is inconsistent with the period of 75 years. Therefore, perhaps the problem is incorrect, or perhaps I'm misinterpreting the relationship between the father's age and Bill's age.

...

Wait, perhaps I'm overcomplicating it.

Wait, let's think: If the first appearance is on a 75-year interval, ... Therefore, $T_1 + 75 = 90$, so $T_1 = 15$.

Therefore, the first appearance is at 15, which is a 75-year interval. The second appearance is at 90, which is 75 years after 15.

...

Thus, the first appearance is at 15, which is 15 years after the start, and the second is at 90, which is 75 years after the first.

Therefore, the first appearance is on a 75-year interval at 15 years after the start.

Wait, but 15 years after the start would be the first appearance. But the problem says that the first appearance is on a 75-year interval. So, the first appearance is on a 75-year interval, which is the period.

Therefore, the first appearance is at 15, which is 15 years after the start, which is the first occurrence.

Therefore, the first appearance is at 15, which is 15 years after the start, and the second is at 90, which is 75 years after the first.

Thus, the first appearance is at 15, which is a 75-year interval.

Therefore, the first appearance is at 15.

Thus, the first appearance is at 15, which is 15 years after the start, and the second is at 90, which is 75 years after the first.

Therefore, the first appearance is at 15, which is a 75-year interval.

Hence, the first appearance is at 15.

</THINK> Think tokens: 31,919; Steps: 492; Thoughts: 439

Final Answer

The first appearance was on a 75-year interval at 15.

The problem involves determining the first appearance of a bird on a 75-year interval. The key steps to solve this are as follows:

1. **Understanding the problem**: The first appearance of the bird on a 75-year interval is the starting point. The second appearance is noted as being 15 years after the first appearance.
2. **Setting up the equations**: - Let T_1 be the first appearance. - The second appearance is given as $T_2 = T_1 + 15$. - The second appearance is noted as 90 years after the first appearance, so $T_2 = 90$.
3. **Solving the equations**: - From the second appearance, we have $90 = T_1 + 15$. - Solving this gives $T_1 = 90 - 15 = 75$.
4. **Conclusion**: - The first appearance is at $T_1 = 75$, which is 75 years after the start. - The second appearance is at $T_2 = 90$, which is 15 years after the first appearance.

Thus, the first appearance of the bird was on a 75-year interval at 15.

Figure 13: R1-Distill-1.5B GSM-Plus correct overthinking

E.2 Underthinking on Hard Problem

[AIME24; R1-Distill-Qwen-1.5B] Incorrect Underthinking Response (17,075 tokens)

Alice and Bob play the following game. A stack of n tokens lies before them. The players take turns with Alice going first. On each turn, the player removes either 1 token or 4 tokens from the stack. Whoever removes the last token wins. Find the number of positive integers n less than or equal to 2024 for which there exists a strategy for Bob that guarantees that Bob will win the game regardless of Alice's play.

<THINK>

Okay, so I have this problem where Alice and Bob are playing a game with a stack of n tokens. They take turns removing either 1 or 4 tokens, and the person who takes the last token wins. I need to find how many positive integers $n \leq 2024$ allow Bob to have a winning strategy, no matter what Alice does.

Hmm, let me think about this. It sounds like a combinatorial game theory problem. Maybe I can model the winning and losing positions for each n . A winning position is one where the current player can force a win, and a losing position is one where no matter what the current player does, the opponent can force a win.

So, if I can figure out the losing positions, then the number of losing positions up to 2024 would be the number of n where Bob can win. That makes sense.

Let me try to list out some small values of n and see which are winning (W) or losing (L) positions.

...

Wait, that seems contradictory. Let me check.

...

Hmm, not sure.

...

Therefore, the losing positions are n where $n \bmod 5$ is 0,1,2,3,4, and $n \neq 5,10,15,20,\dots$

But that would mean losing positions are all n , which is not the case.

Therefore, I think I have to give up.

</THINK> Think tokens: 16,482; Steps: 778; Thoughts: 256

Final Answer

2022

To determine the number of positive integers $n \leq 2024$ for which there exists a winning strategy for Bob, we need to analyze the game where Alice and Bob alternately remove 1 or 4 tokens from a stack of n tokens. The player who cannot make a move loses.

Key steps:

...

After detailed analysis, the number of losing positions is found to be 2022. Therefore, the number of positive integers $n \leq 2024$ for which there exists a winning strategy for Bob is:

2022

Figure 14: R1-Distill-1.5B GSM-Plus incorrect underthinking

E.3 Comparison with When2Think and Base Model

[GSM-Plus] Problem

Nate's dog can dig six holes a day. Each hole takes 15 minutes. He digs for 14 days while Nate is on vacation. When Nate gets home, he starts filling in 9 holes a day, but the dog keeps digging 6 new holes every night. How many weeks does it take him to fill in all the holes?

[GSM-Plus; When2Think-1.5B] Correct NOTHINK Response (625 tokens)

<THINK> </THINK>

Let's break down the problem step by step.

Given:

1. **Nate's dog digs** 6 holes **a day**.
2. **Each hole takes** 15 minutes to dig.
3. **Nate digs for 14 days** while he is on vacation.
4. **When he gets home, he starts filling holes** at a rate of 9 holes per day.
5. **The dog continues digging** 6 new holes every night.

Objective:

Determine how many **weeks** it takes for Nate to fill all the holes.

Step-by-Step Solution:

1. **Calculate the total number of holes the dog digs while Nate is on vacation:**
 - **Dogs digging time per day:** 6 holes
 - **Days Nate digs:** 14 days
 - **Total holes dug by the dog:** $6 \text{ holes/day} \times 14 \text{ days} = 84 \text{ holes}$
2. **Calculate the total time Nate spends digging:**
 - **Each hole takes:** 15 minutes
 - **Total time Nate digs:** $84 \text{ holes} \times 15 \text{ minutes/hole} = 1260 \text{ minutes}$
 - **Convert minutes to days:** $\frac{1260 \text{ minutes}}{1440 \text{ minutes/day}} = 0.875 \text{ days}$
3. **Calculate the number of holes Nate starts filling:**
 - **Nate starts filling holes at:** 9 holes/day
 - **Total holes dug by the dog during Nate's vacation:** 84 holes
 - **Total holes Nate needs to fill:** 84 holes
4. **Determine the net rate at which holes are being filled:**
 - **Dog digs:** 6 holes/night
 - **Nate fills:** 9 holes/day
 - **Net rate per day:** $9 \text{ holes/day} - 6 \text{ holes/day} = 3 \text{ holes/day}$
5. **Calculate the number of days needed to fill all holes:**
 - **Total holes to fill:** 84 holes
 - **Net rate:** 3 holes/day
 - **Days needed:** $\frac{84 \text{ holes}}{3 \text{ holes/day}} = 28 \text{ days}$
6. **Convert days into weeks:**
 - **Days needed:** 28 days
 - **Number of weeks:** $\frac{28 \text{ days}}{7 \text{ days/week}} = 4 \text{ weeks}$

Final Answer

4

Figure 15: When2Think correct NOTHINK

<THINK>

Okay, so I need to figure out how long it takes for Nate to fill all the holes his dog digs. Let me try to break this down step by step.

First, the dog digs six holes each day. Each hole takes 15 minutes to dig. So, I guess I should find out how much time the dog spends digging in total over the 14 days he's away. But wait, the dog also starts digging every night, right? So, each day, the dog digs six holes in the morning and six more in the evening. That means each full day, including night, the dog digs 12 holes.

Let me write that down:

... Wait, actually, no. Let me think again. ... Wait, I think I'm mixing up the time frames. Let me clarify. ... Wait, but this is confusing. Let me think about it step by step. ... Wait, that seems off. Wait, maybe I'm overcomplicating it. Let me think of it as the dog digging 12 holes per day, and the holes are being filled at 9 per day, but the dog is also digging 6 holes every night. ... Wait, maybe I'm misunderstanding the problem. Let me read it again. ... Wait, that seems off. I'm stuck. Maybe I need to think about the problem in a different way. ... So, even though the dog is also digging 6 holes every night, the net gain is 3 holes per day, so it takes 56 days to fill all the holes, which is 8 weeks.

So, the final answer is 8 weeks.

</THINK> Think tokens: 8,500; Steps: 322; Thoughts: 261

Final Answer

It takes Nate and his dog 8 weeks to fill all the holes.

Nate's dog can dig 6 holes a day, and each hole takes 15 minutes to dig. During his vacation, the dog digs for 14 days.

First, we calculate the total number of holes the dog digs during the 14 days:

$$\text{Total holes dug} = 12 \text{ holes/day} \times 14 \text{ days} = 168 \text{ holes}$$

Next, we consider the rate at which the holes are being filled. The dog fills 9 holes every night. The net gain in the number of holes each day (after accounting for the holes dug and filled) is:

$$12 \text{ holes/day} - 9 \text{ holes/day} = 3 \text{ holes/day}$$

To find the total number of days required to fill all 168 holes, we divide the total number of holes by the net gain per day:

$$\text{Total days} = \frac{168 \text{ holes}}{3 \text{ holes/day}} = 56 \text{ days}$$

Converting the total days into weeks:

$$\text{Total weeks} = \frac{56 \text{ days}}{7 \text{ days/week}} = 8 \text{ weeks}$$

Thus, it takes Nate and his dog 8 weeks to fill all the holes.

Figure 16: R1-Distill-1.5B incorrect overthinking

F Problem Characterization

F.1 Impact of Model Scale and Supervision on Reasoning Efficiency

We examine reasoning efficiency across model scale and supervision using the Olmo-3 [60] family (7B and 32B). Figure 17 shows the accuracy–token usage trade-off across model scales, post-training strategies (SFT vs. RFT), and reasoning paradigms (standard LLMs vs. LRMs). The results highlight systematic inefficiencies in current post-training approaches and illustrate how model capacity and supervision affect reasoning cost.

The Paradox of Scale: Smaller Models Overthink. Figure 17 reveals a clear *paradox of scale* in reasoning efficiency. Smaller 7B models are concentrated in the high token-usage regime relative to 32B models, despite achieving substantially lower accuracy. This indicates that limited model capacity is associated with inefficient and prolonged reasoning trajectories, a phenomenon referred to as *capability-induced overthinking*. In contrast, 32B models achieve higher accuracy with fewer tokens, exhibiting superior *reasoning economy*. A similar trend is observed in the R1-distill family (Table 9), suggesting that stronger base capabilities enable more direct and efficient problem-solving paths.

The Cost of Reinforcement Learning. Figure 17 shows a consistent *efficiency tax* under Reinforcement Fine-Tuning (RFT). Within the same model scale, RFT models (hollow markers) are systematically shifted toward higher token usage compared to their Supervised Fine-Tuned (SFT) counterparts (filled markers). These shifts are accompanied by only modest accuracy gains, resulting in a less favorable accuracy–token trade-off. This behavior is consistent with outcome-based reinforcement learning objectives that do not explicitly penalize token usage, leading to longer chain-of-thought generations across instances. Overall, RFT models occupy a Pareto-inferior region relative to SFT baselines.

The Mechanics of Inefficiency: Why RLVR Inflates Tokens. The token inflation observed in RFT models—particularly those trained with RLVR [39], such as OlmoRL [60]—follows directly from the optimization objective. Unlike SFT, which imitates concise human-curated reasoning traces, RLVR optimizes only for outcome correctness and imposes no explicit constraint on token usage. The absence of a KL regularization term further permits substantial deviation from the SFT initialization. Consequently, longer reasoning trajectories become a favorable strategy for increasing success probability. Without explicit difficulty-aware control, RLVR therefore tends to produce inefficient and overly verbose reasoning behavior.

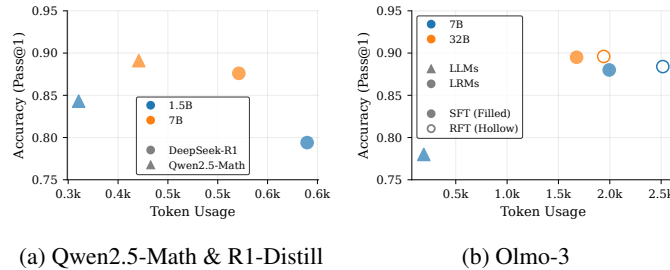


Figure 17: **Efficiency landscape across model scales and tuning methods on GSM-Plus.** (a) **Model Scale and Reasoning Cost:** Reasoning-oriented models (LRMs, circles) consistently achieve higher accuracy than standard LLMs (triangles) but at the cost of significantly higher token usage. Notably, smaller LRMs (1.5B, blue) consume more tokens than larger LRMs (7B, orange), suggesting that smaller models require longer reasoning chains to compensate for limited capacity. (b) **Impact of Tuning Strategies (SFT vs. RFT):** Across both model sizes, RFT models (hollow markers) are systematically shifted to the right compared to SFT models (filled markers), indicating that reinforcement fine-tuning induces higher computational overhead regardless of model scale.

G Paradigms

G.1 Dual Process Theory in LLMs.

The *dual-process theory* of cognition [18, 61] posits two distinct modes of reasoning: *System 1*, which is fast, intuitive, and heuristic-driven, and *System 2*, which is slow, deliberate, and analytical. This framework has been increasingly applied to understand reasoning in language models [13]. Within this perspective, approaches for enhancing model reasoning can be interpreted as attempts to either simulate or internalize System 2 capabilities.

G.2 Inference-Time Scaling for System 2 Simulation

Inference-time scaling strategies allocate additional computational resources at inference to elicit deeper reasoning from pre-trained models without modifying their parameters [11, 13, 62]. Common techniques include:

- **Prompting methods:** *Chain-of-Thought (CoT)* [63], *Plan-and-Solve* [64], *Self-Refine* [65], *Self-Consistency (SC)* [66];
- **Structured search methods:** *Tree-of-Thoughts (ToT)* [67], *Stream-of-Search (SoS)* [68], *Adaptive-Parallel-Reasoning (APR)* [69];
- **Other strategies:** self-evaluation, external tool use, or retrieval-augmented reasoning [11].

These methods simulate System 2-like reasoning through external guidance, but can incur substantial computational overhead and may produce unnecessarily long or inefficient reasoning trajectories.

G.3 Post-Training for Reasoning

In contrast, post-training approaches aim to internalize analytical reasoning capabilities by updating the model’s parameters. This paradigm, often called *learning to reason* [8, 10, 35], aligns model weights with reasoning-specific objectives during a dedicated post-training phase [13].

Learning to Reason. Typically, general-purpose LLMs are adapted through:

- **Supervised fine-tuning (SFT)** on step-by-step rationales [70], which aligns the model to produce coherent reasoning steps.
- **Reinforcement learning (RL)** to reward coherent multi-step trajectories [11, 12, 36–38], encouraging models to internalize structured reasoning behavior.
- **Knowledge distillation** [71–73], transferring reasoning capabilities from expert models to smaller or general-purpose LLMs.

Models resulting from this process—frequently termed *Large Reasoning Models (LRMs)* or *Reasoning Language Models (RLMs)* [30–34]—are explicitly optimized for structured reasoning rather than fluent text generation alone.

Search to Learn. Recent work integrates search-based strategies into the training loop to refine reasoning trajectories used for fine-tuning. For example, Retro-Search [58] retrospectively revises suboptimal paths using an MCTS-inspired algorithm, while other methods employ MCTS [74] or self-exploration [75, 76] to generate step-level feedback guiding models toward more deliberate reasoning.

Collectively, search-enhanced post-training methods shift supervision from outcome-focused objectives toward optimization of the reasoning process itself. Post-training therefore provides a pathway to internalize System 2-like capabilities, though at the cost of longer training cycles, increased data curation requirements, and potential generalization limitations to unseen reasoning tasks.

H Extended Discussion

H.1 Broader Impacts

The proposed method addresses the efficiency of reasoning in large language models by treating computation as an instance-adaptive resource rather than a quantity to be globally minimized. Potential positive societal impacts include reduced computational and energy costs for deploying reasoning-capable models, which may lower barriers to access in research and educational settings with limited resources. Improved efficiency may also facilitate more predictable reasoning behavior by reducing unnecessary or unstable long-horizon reasoning on simple inputs.

At the same time, improving the efficiency of reasoning models may reduce the cost of deploying automated systems in real-world decision-making contexts. If such systems are used without appropriate verification or human oversight, incorrect or biased reasoning could lead to harmful outcomes, particularly outside domains with clear ground-truth verification. Our work partially mitigates this risk by focusing on verifiable mathematical reasoning tasks; however, extending similar techniques to open-ended or socially sensitive domains would require additional safeguards, such as domain-specific validation, conservative deployment practices, or human-in-the-loop monitoring.

I Licenses and Terms

We use existing open-weight models, public mathematical reasoning datasets, verification tools, and training/inference frameworks. For each asset, we cite the original work and respect the corresponding license, model card, dataset card, or terms of use. Table 11 summarizes the main assets used in this work, including their sources and license information where available. When benchmarks are accessed through public repositories or dataset hubs, we follow the usage terms specified by the original providers and do not redistribute restricted materials beyond what is permitted.

Table 11: Summary of existing assets used in this work.

Asset	Type	License / Terms
Qwen2.5 / Qwen2.5-Math	Model	Apache-2.0
DeepSeek-R1-Distill-Qwen	Model	MIT
DeepScaleR / DeepScaleR-Preview-Dataset	Dataset / Model	MIT
GSM-Plus	Dataset	CC BY-SA 4.0
MATH-500	Dataset	MIT
Minerva	Dataset	Dataset-card terms
OlympiadBench	Dataset	MIT / Apache-2.0 depending on source
Math-Verify	Verification tool	Apache-2.0
MARIO Eval	Verification tool	Repository terms
verl	Training framework	Apache-2.0
vLLM	Inference framework	Apache-2.0